

DISTRICARNES

Entender tu proyecto paso a paso

Modelo · Vista · Controlador

Qué hace el sistema, dónde están sus botones y cómo se conectan sus archivos.

44 capturas de código + 10 pantallas del proyecto con datos ilustrativos.

Empieza por la explicación en lenguaje natural. Después consulta la referencia de funciones y sigue el código en tu editor.

Material de estudio · 6 de septiembre de 2026

Cómo estudiar con este manual

1. Lee qué responsabilidad tiene el archivo dentro de MVC.
2. Compara la captura con el archivo en tu editor: sus números de línea son reales.
3. Sigue quién lo llama, qué recibe y qué devuelve. Después prueba explicarlo sin mirar.
4. Distingue el funcionamiento actual de las mejoras propuestas. No cambies varios archivos a la vez durante una práctica.

Mapa de lectura

1. ¿De qué se trata tu proyecto?
2. Cómo entender MVC sin perderte en los nombres
3. Primera pantalla: entrar al sistema
4. El panel: dónde encuentras las funcionalidades
5. Inventario: consultar y cambiar productos
6. Nueva venta: del producto seleccionado al cobro
7. Historial, recibo e impresión
8. Administradores: quién puede usar el sistema
9. Las categorías también recorren varios archivos
10. Los archivos que conectan todo
11. ¿Qué pasa al encender el proyecto?
12. Dónde está la apariencia y por qué se mueven los botones
13. La base de datos explicada como información del negocio
14. Los demás archivos: documentación y herramientas
15. Un ejemplo completo para explicárselo al profesor
16. Referencia detallada de funciones y bloques

Referencia 1. Archivos de raíz y configuración

Referencia 2. Arranque y configuración del backend

Referencia 3. Las seis rutas: el mapa entre URL y controlador

Referencia 4. Middleware: comprobaciones compartidas

Referencia 5. Servicios

Referencia 6. Controladores: contenido completo por función

Referencia 7. Modelos: qué significa cada consulta

Referencia 8. FRONTEND: los dos documentos HTML

Referencia 9. FRONTEND/js/api.js: comunicación común

Referencia 10. FRONTEND/js/ui.js: utilidades visuales

Referencia 11. FRONTEND/js/login.js

Referencia 12. FRONTEND/js/app.js: coordinación del panel

Referencia 13. FRONTEND/js/dashboard.js

Referencia 14. FRONTEND/js/inventory.js

Referencia 15. FRONTEND/js/administrators.js

Referencia 16. FRONTEND/js/sales.js: cada función y cada evento

Referencia 17. FRONTEND/css/styles.css: bloques y relación con HTML/JS

Referencia 18. FRONTEND/images: los cinco archivos

Referencia 19. BACKEND/database/schema.sql: cada bloque, tabla y regla

Referencia 20. build_integracion_tests.mjs y outputs

Referencia 21. Tres recorridos completos con los archivos conectados

Referencia 22. Arquitectura objetivo: cambios concretos y graduales

1. ¿De qué se trata tu proyecto?

Districarnes es una aplicación para que una persona que atiende una carnicería pueda consultar los productos disponibles, actualizar el inventario y registrar lo que vende. También permite revisar ventas anteriores, imprimir comprobantes y crear cuentas para otras personas que van a operar el sistema. Su propósito es organizar esas tareas en un solo lugar.

Imagina una situación de trabajo: llega un cliente y pide 400 gramos de Punta de Anca. La persona que atiende busca el producto, lo agrega al carrito, indica la cantidad y registra cómo pagó el cliente. El sistema calcula cuánto debe cobrar, comprueba las existencias, guarda la venta y disminuye el inventario. Finalmente muestra el recibo. Esa situación resume la operación más importante del proyecto.

El inventario y las ventas están conectados. Un producto creado en Inventario puede aparecer en Nueva venta cuando está activo y tiene existencias. Una venta registrada aparece en Historial y afecta las cifras de resumen. Por eso los módulos no son pantallas independientes sin relación: comparten productos, categorías, cuentas y ventas.

El proyecto no hace todo lo que podría hacer un negocio. No se encontraron compras a proveedores, contabilidad completa, devoluciones operativas, pagos bancarios conectados ni facturación electrónica integrada. Seleccionar Tarjeta o Transferencia registra el método elegido; no significa que la aplicación consulte un banco. Esto es importante para explicar con precisión qué desarrollaste.

Las capturas de pantalla de esta edición utilizan el HTML, CSS y JavaScript reales del proyecto con datos ilustrativos suministrados para el manual. No representan el estado actual de tu base ni son evidencia de ventas ejecutadas. Los círculos numerados se agregaron a las capturas para ayudarte a ubicar controles. Las capturas de código sí reproducen fragmentos de tus archivos, con sus números de línea.

2. Cómo entender MVC sin perderte en los nombres

La Vista es lo que la persona ve y utiliza: el formulario de acceso, el menú, la tabla de inventario, los campos y los botones. En tu proyecto se construye en FRONTEND. El HTML coloca los elementos, el CSS les da presentación y el JavaScript del navegador responde a lo que hace la persona.

El Controlador recibe una solicitud de la Vista. Por ejemplo: «quiero crear este producto con este nombre, este precio y este stock». Examina los datos y decide si puede continuar. Si faltan datos o son inválidos, devuelve un mensaje. Si son válidos, pide al Modelo que haga la operación. Tus controladores están en BACKEND/src/controllers.

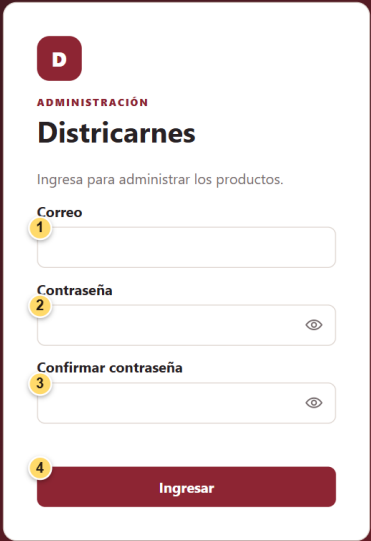
El Modelo se encarga de las operaciones sobre los datos y parte de las reglas del negocio. Sus archivos están en BACKEND/src/models y utilizan MySQL. Guardar un producto significa insertar una fila en una tabla; listar productos significa consultarlas. Los servicios y los triggers apoyan las reglas y operaciones del Modelo y de la aplicación.

Las rutas no son otra pantalla. Son direcciones que permiten llegar a una operación. Por ejemplo, /api/productos identifica operaciones relacionadas con productos. El método GET pide consultar; POST pide crear; PUT pide actualizar; DELETE pide eliminar. La misma dirección puede tener comportamientos distintos porque cambia el método.

Para estudiar una función, no empieces preguntando «¿qué significa cada símbolo?». Empieza preguntando «¿qué tarea está resolviendo?». Después identifica qué recibe, qué hace con eso y qué entrega. Finalmente conecta sus instrucciones con esa tarea. Esta forma de leer te permite comprender el código aunque el profesor cambie su orden o el nombre de una variable.

3. Primera pantalla: entrar al sistema

PANTALLA 01 · Interfaz real con datos ilustrativos

La imagen muestra una interfaz de usuario para el sistema 'Districarnes'. El fondo es un color rojo oscuro. En el centro hay un formulario blanco con un logo 'D' en un cuadrado rojo en la parte superior izquierda. Debajo del logo, el texto 'ADMINISTRACIÓN' aparece en rojo y 'Districarnes' en negro. Una instrucción dice 'Ingresa para administrar los productos.' Hay tres campos de entrada: 'Correo' (marcado con un 1), 'Contraseña' (marcado con un 2 y un ícono de ojo a la derecha) y 'Confirmar contraseña' (marcado con un 3 y un ícono de ojo a la derecha). En la parte inferior del formulario hay un botón rojo con el texto 'Ingresar' (marcado con un 4).

En la captura, el número 1 identifica el correo; el 2, la contraseña; el 3, la confirmación de contraseña; y el 4, el botón Ingresar. Los ojos a la derecha de las claves permiten ver u ocultar lo escrito. No cambian la contraseña: solo cambian cómo se muestra el campo.

FRONTEND/index.html: dónde se construye el acceso

CAPTURA 27 · FRONTEND/index.html

FRONTEND/index.html

CÓDIGO ORIGINAL

```

10  <main class="login-page">
11    <form id="login-form" class="card login-card" autocomplete="off">
12      <div class="logo">D</div>
13      <p class="overline">Administración</p>
14      <h1>Districarnes</h1>
15      <p class="muted">Ingresa para administrar los productos.</p>
16
17      <label>
18        Correo
19        <input id="email" name="login_identifier" type="email" autocomplete="off" autocapitalize="none"
        spellcheck="false" data-lpignore="true" data-1p-ignore required>
20      </label>
21      <label>
22        Contraseña
23        <span class="password-field">
24          <input id="password" name="login_secret" type="password" autocomplete="new-password" data-
          lpignore="true" data-1p-ignore required>
25          <button type="button" class="password-eye" data-password-target="password" aria-
          label="Mostrar contraseña" title="Mostrar contraseña">
26            <svg viewBox="0 0 24 24" aria-hidden="true"><path d="M2 12s3.5-6 10-6 10 6 10 6-3.5 6-10
          6s2 12z"></path><circle cx="12" cy="12" r="3"></circle></svg>
27          </button>

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Este archivo contiene la primera pantalla. Cuando escribes la dirección principal del sistema, el servidor entrega este documento al navegador. Dentro aparece un formulario llamado login-form. En ese formulario se encuentran los campos email, password y password-confirmation, además del botón Ingresar.

Los nombres que aparecen después de id funcionan como identificadores: permiten que JavaScript encuentre un elemento concreto. El texto visible del botón puede decir «Ingresar», mientras que el programa lo reconoce como el botón de envío del formulario. Cambiar únicamente el texto visible no tiene el mismo efecto que cambiar un id usado por JavaScript.

Al final del HTML se carga login.js. Esa es la conexión entre la pantalla dibujada y su comportamiento. El HTML por sí solo no comprueba si una cuenta está registrada en MySQL. Su responsabilidad es ofrecer los campos y las restricciones básicas, como pedir que el correo tenga forma de correo y que los campos obligatorios no estén vacíos.

FRONTEND/js/login.js: qué ocurre al usar el acceso

CAPTURA 31 · FRONTEND/js/login.js

FRONTEND/js/login.js

CÓDIGO ORIGINAL

```
1 import { hasSession, startSession } from './api.js';
2
3 if (hasSession()) {
4   window.location.replace('/app');
5 }
6
7 const form = document.querySelector('#login-form');
8 const errorMessage = document.querySelector('#login-error');
9 const submitButton = form.querySelector('button[type="submit"]');
10 const emailInput = document.querySelector('#email');
11 const passwordInput = document.querySelector('#password');
12 const passwordConfirmationInput = document.querySelector('#password-confirmation');
13 const passwordEyes = document.querySelectorAll('[data-password-target]');
14
15 function clearLoginFields() {
16   form.reset();
17   emailInput.value = '';
18   passwordInput.value = '';
19   passwordConfirmationInput.value = '';
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Este archivo localiza el formulario y sus campos. `clearLoginFields` deja los campos vacíos, vuelve a ocultar las contraseñas y limpia el mensaje de error. Por eso, si el profesor pregunta quién limpia los datos de acceso, esta función es el punto de partida.

Luego prepara los botones del ojo. Cuando haces clic, lee a qué campo apunta el botón y alterna entre mostrar texto y ocultarlo como contraseña. Los atributos `data-password-target` del HTML son el vínculo que le indica cuál campo debe cambiar.

La acción más importante está en el evento `submit` del formulario. Un evento es algo que sucede, como hacer clic, escribir o enviar un formulario. Aquí no se llama una función con un nombre propio como `guardarLogin`: se registra una función directamente dentro de `addEventListener`. Aunque no tenga nombre, sigue siendo una función y contiene comportamiento que debes estudiar.

Al enviar, evita la recarga tradicional, compara las dos claves y muestra un error si son diferentes. Si coinciden, deshabilita temporalmente Ingresar y llama a `startSession`. Cuando el servidor confirma el acceso, abre `/app`. Si el servidor rechaza los datos, muestra el error y vuelve a habilitar el botón. La confirmación de clave solo se compara en esta pantalla; la API recibe correo y una contraseña.

BACKEND/src/routes/authRoutes.js: cómo llega el acceso al servidor

CAPTURA 9 · BACKEND/src/routes/authRoutes.js

BACKEND/src/routes/authRoutes.js

CÓDIGO ORIGINAL

```

1 const express = require('express');
2 const authController = require('../controllers/authController');
3 const { requireAuth } = require('../middleware/authMiddleware');
4
5 const router = express.Router();
6
7 router.post('/login', authController.login);
8 router.post('/logout', requireAuth, authController.logout);
9
10 module.exports = router;
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Aquí se registra POST /login y se conecta con authController.login. El prefijo /api se agrega en otro archivo, por eso la dirección completa que utiliza el navegador es POST /api/login. Esta ruta permite entrar sin tener una sesión anterior: exigir una sesión para iniciar la primera sesión impediría acceder.

También se registra POST /logout, que sí exige sesión. Su tarea es cerrar el acceso que ya existe. No confundas la ruta /login con el archivo login.js: la primera es una dirección HTTP del servidor y el segundo es un archivo que se ejecuta en el navegador.

BACKEND/src/controllers/authController.js: decidir si puede ingresar

CAPTURA 19 · BACKEND/src/controllers/authController.js

BACKEND/src/controllers/authController.js

CÓDIGO ORIGINAL

```

1 const administratorModel = require('../models/administratorModel');
2 const sessionService = require('../services/sessionService');
3
4 async function login(req, res, next) {
5   try {
6     const email = req.body.email?.trim().toLowerCase();
7     const password = req.body.password;
8     if (!email || !password) {
9       return res.status(400).json({ message: 'Correo y contraseña son obligatorios' });
10    }
11
12    const administrator = await administratorModel.findActiveByCredentials(email, password);
13    if (!administrator) {
14      return res.status(401).json({ message: 'Credenciales incorrectas' });
15    }
16
17    res.json({ token: sessionService.create(administrator) });
18  } catch (error) {
19    next(error);
20  }
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

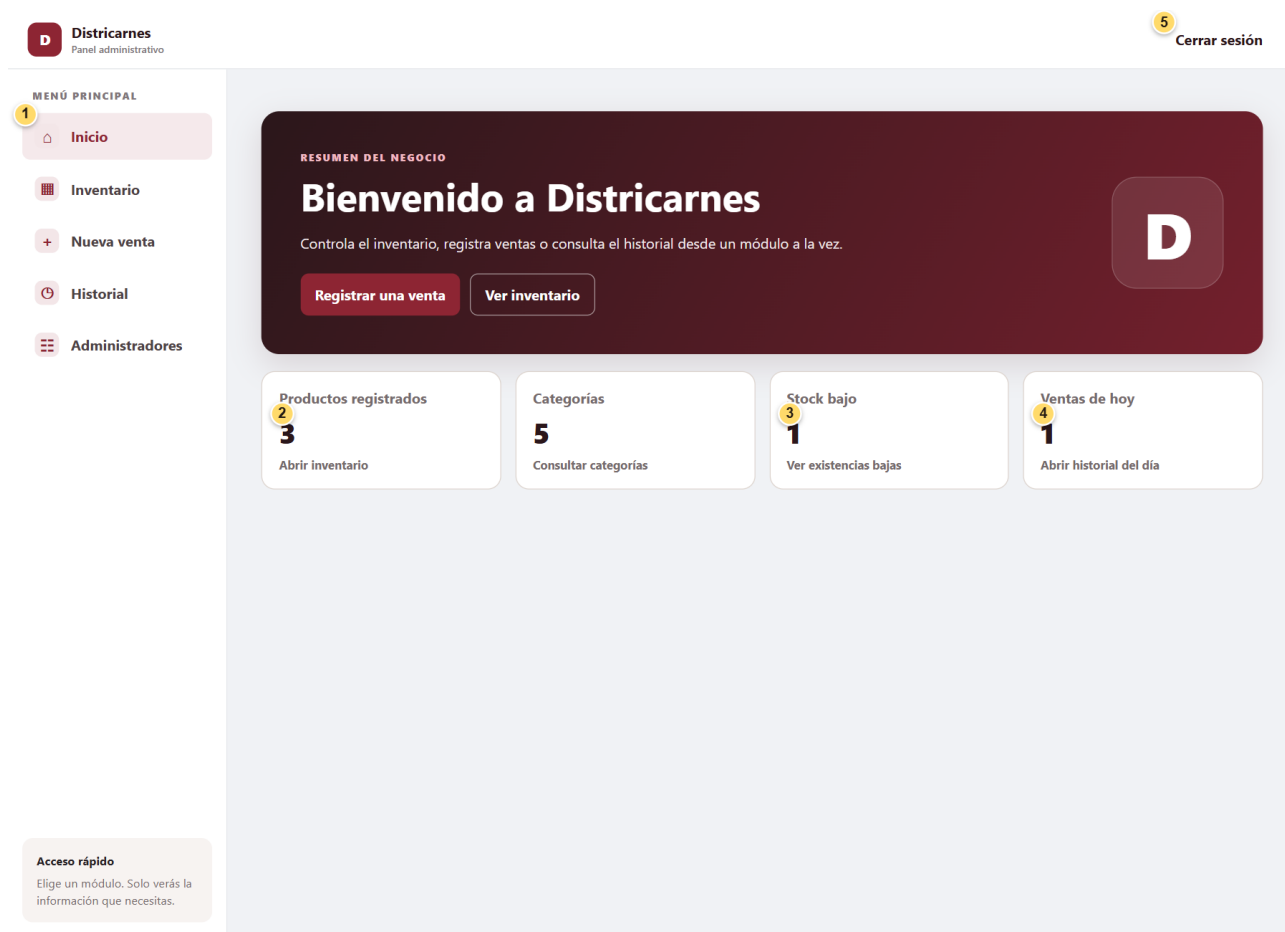
login recibe el correo y la contraseña. Limpia los espacios del correo, lo pasa a minúsculas y comprueba que ambos datos estén presentes. Después pide a administratorModel que busque una cuenta activa

con esas credenciales. Si no existe, responde «Credenciales incorrectas». Si existe, crea una sesión y envía un token al navegador.

Un token es un identificador que el navegador presenta en las siguientes solicitudes para que el servidor reconozca la sesión. No necesitas volver a mandar la contraseña al consultar cada pantalla. logout hace la operación inversa: elimina la sesión reconocida por ese token y devuelve el mensaje de cierre.

4. El panel: dónde encuentras las funcionalidades

PANTALLA 02 · Interfaz real con datos ilustrativos



En el número 1 está el menú: Inicio, Inventario, Nueva venta, Historial y Administradores. Los números 2, 3 y 4 señalan tarjetas de resumen que también se pueden pulsar. El número 5 identifica Cerrar sesión, arriba a la derecha. La ubicación descrita corresponde a la vista de escritorio; en una pantalla pequeña el CSS reorganiza el menú.

FRONTEND/app.html: la estructura del panel completo

CAPTURA 28 · FRONTEND/app.html

FRONTEND/app.html

CÓDIGO ORIGINAL

```

14
15 <div class="app-shell">
16   <aside class="sidebar">
17     <p class="menu-label">Menú principal</p>
18     <nav class="side-nav" aria-label="Navegación principal">
19       <button data-section="home" class="nav-button active"><span>🏠</span> Inicio</button>
20       <button data-section="inventory" class="nav-button"><span>📦</span> Inventario</button>
21       <button data-section="sales" class="nav-button"><span>+</span> Nueva venta</button>
22       <button data-section="history" class="nav-button"><span>📜</span> Historial</button>
23       <button data-section="administrators" class="nav-button"><span>👤</span>
    Administradores</button>
24     </nav>
25     <div class="sidebar-help"><strong>Acceso rápido</strong><p>Elige un módulo. Solo verás la
    información que necesitas.</p></div>
26   </aside>
27
28   <main class="workspace">
29     <p id="message" class="message" aria-live="polite"></p>
30     <section id="home-section">
31       <article class="welcome-card">

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Este documento contiene las cinco secciones del panel. Al cambiar de menú no estás abriendo cinco archivos HTML diferentes. Sigues dentro de app.html y JavaScript oculta unas secciones y muestra otra. Por eso puedes conservar algunos filtros y el carrito mientras navegas sin recargar la página.

Cada botón del menú tiene un atributo data-section, por ejemplo inventory. La sección correspondiente se llama inventory-section. Esa coincidencia permite relacionar el botón «Inventario» con el bloque que debe verse. La clase hidden hace que una sección quede oculta, pero no elimina sus elementos del documento.

Aquí también están los formularios emergentes llamados dialog: uno de producto, otro de administrador, otro para el recibo y otro para el reporte. No son páginas nuevas. Una función los abre sobre el panel y otra los cierra. Si buscas el botón Guardar producto, debes buscar dentro de product-form; si buscas Imprimir recibo, dentro de receipt-dialog.

Algunos botones no aparecen escritos directamente en app.html. Editar y Eliminar se repiten una vez por producto y los crea inventory.js al construir cada fila. Agregar lo crea sales.js para cada tarjeta del catálogo. Esto explica por qué buscar la palabra del botón solo en HTML a veces no lo encuentra.

FRONTEND/js/app.js: quién hace funcionar el menú

CAPTURA 32 · FRONTEND/js/app.js

FRONTEND/js/app.js

CÓDIGO ORIGINAL

```

1 import { apiRequest, closeSession, hasSession } from './api.js';
2 import { initializeAdministrators, loadAdministrators } from './administrators.js';
3 import { loadDashboard } from './dashboard.js';
4 import { initializeInventory, loadInventory, selectInventoryCategory, selectLowStock } from
  './inventory.js';
5 import { initializeSales, loadSaleCatalog, loadSalesHistory } from './sales.js';
6 import { $, showMessage } from './ui.js';
7
8 const sectionLoaders = {};
9
10 async function openSection(name) {
11   document.querySelectorAll('.side-nav [data-section]').forEach(button => {
12     button.classList.toggle('active', button.dataset.section === name);
13   });
14
15   Object.keys(sectionLoaders).forEach(section => {
16     $('#${section}-section').classList.toggle('hidden', section !== name);
17   });
18
19   try {

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

`initializeApplication` prepara el panel: pide las categorías al servidor, se las entrega a inventario y ventas, prepara administradores y conecta los botones. Después abre Inicio. Inicializar significa dejar listos los eventos y controles; cargar significa traer o actualizar los datos que se mostrarán.

`openSection` recibe el nombre de la sección elegida. Marca el botón del menú como activo, oculta las otras secciones y ejecuta la función que carga la seleccionada. `sectionLoaders` es la relación entre cada sección y esa función. Por ejemplo, `inventory` corresponde a `loadInventory` y `home` a `loadDashboard`.

El archivo escucha clics en elementos con `data-section`. Si pulsas una tarjeta que solicita stock bajo, primero ajusta ese filtro y luego abre Inventario. También conecta el botón `logout` con `closeSession`. Esta es la razón por la que modificar la navegación normalmente empieza aquí y en los atributos de `app.html`.

FRONTEND/js/dashboard.js: de dónde salen los números de Inicio

CAPTURA 33 · FRONTEND/js/dashboard.js

FRONTEND/js/dashboard.js

CÓDIGO ORIGINAL

```
1 import { apiRequest } from './api.js';
2 import { $, today } from './ui.js';
3
4 export async function loadDashboard(categories) {
5   const date = today();
6   const [products, sales] = await Promise.all([
7     apiRequest('/productos'),
8     apiRequest(`/ventas?fecha=${date}`)
9   ]);
10
11   $('#stat-products').textContent = products.length;
12   $('#stat-categories').textContent = categories.length;
13   $('#stat-low-stock').textContent = products.filter(product => product.stock <= 10).length;
14   $('#stat-sales').textContent = sales.length;
15 }
```

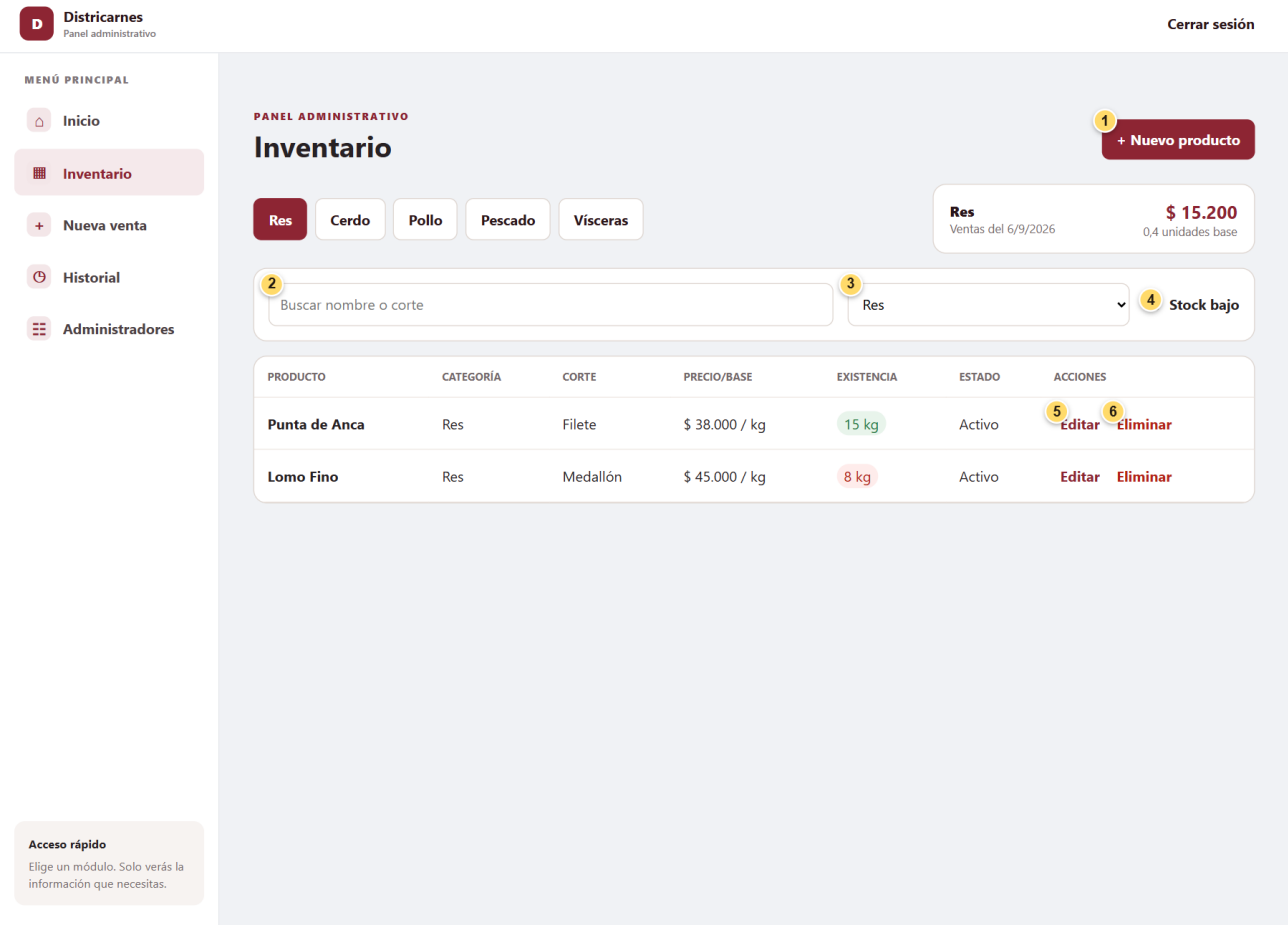
Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

loadDashboard consulta la lista de productos y las ventas de la fecha actual. Cuenta cuántos productos recibió, cuántas categorías hay, cuántos productos tienen stock menor o igual que diez y cuántas ventas se encontraron. Luego escribe esos números en las tarjetas del HTML.

No guarda esas cifras en una tabla llamada dashboard. Las calcula cuando carga la pantalla a partir de consultas existentes. Tampoco muestra utilidad del negocio: contar ventas no es calcular ganancias. Los productos inactivos siguen incluidos en el conteo general porque la consulta de productos los devuelve.

5. Inventario: consultar y cambiar productos

PANTALLA 03 · Interfaz real con datos ilustrativos



El botón 1, Nuevo producto, está arriba a la derecha del Inventario. El campo 2 busca por nombre o corte. El selector 3 cambia la categoría y la casilla 4 limita la consulta a stock bajo. Los botones 5 y 6 son Editar y Eliminar: pertenecen a una fila concreta, por eso cada uno debe llevar el identificador de su producto.

Las pestañas con nombres de categorías son otra forma de seleccionar la categoría. El resumen al lado muestra lo vendido ese día para la seleccionada. Ese resumen habla de ventas; la tabla habla de existencias. No debes interpretar el total vendido como el valor del inventario disponible.

FRONTEND/js/inventory.js: las funciones que mueven esta pantalla

CAPTURA 34 · FRONTEND/js/inventory.js

FRONTEND/js/inventory.js

CÓDIGO ORIGINAL

```

1 import { apiRequest } from './api.js';
2 import { $, escapeHtml, formatDate, formatMoney, formatQuantity, showMessage, today } from './ui.js';
3
4 let products = [];
5 let inventoryCategories = [];
6
7 function renderCategoryTabs() {
8   const selectedId = Number($('#category-filter').value);
9   $('#inventory-category-tabs').innerHTML = inventoryCategories.map(category => `
10     <button type="button" class="category-tab ${category.id === selectedId ? 'active' : ''}"
11       data-inventory-category="${category.id}">
12       ${escapeHtml(category.nombre)}
13     </button>
14   `).join('');
15 }
16
17 function renderCategorySummary(summary) {
18   const selectedCategory = inventoryCategories.find(
19     category => category.id === Number($('#category-filter').value)

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

`initializeInventory` recibe las categorías, llena los selectores, dibuja las pestañas y conecta los eventos. Allí el clic de `new-product` llama a `openProductForm` sin pasarle un producto. Esa ausencia significa «vamos a crear uno nuevo». El clic de `close-dialog` cierra la ventana sin guardar.

`openProductForm` se utiliza tanto para crear como para editar. Primero limpia el formulario. Si recibe un producto, coloca sus datos y su ID; si no recibe ninguno, prepara valores iniciales y deja el ID vacío. Finalmente llama a `showModal` para que la ventana aparezca. Abrir la ventana todavía no modifica MySQL.

`productPayload` recoge lo escrito en nombre, corte, categoría, unidad, precio, stock y activo. Lo convierte en un objeto para enviarlo. Piensa en esa función como el momento en que el programa reúne los campos de la ficha: aún no los guarda, solo los organiza.

El evento `submit` de `product-form` realiza el envío. Si `product-id` tiene valor, usa `PUT` y agrega ese ID a la dirección; si está vacío, usa `POST`. Ambos casos aprovechan `productPayload`. Cuando la API confirma, cierra la ventana, muestra el aviso y llama a `loadInventory` para que la tabla refleje el cambio. Si hay error, muestra el mensaje.

`loadInventory` toma los filtros actuales y pide al servidor los productos correspondientes. Puede pedir además el resumen diario de categorías. Cuando llegan los datos, los conserva en `products` y llama a `renderProducts`. Esta separación importa: `loadInventory` consigue la información y `renderProducts` la convierte en filas visibles.

`renderProducts` crea la tabla, incluyendo los botones Editar y Eliminar de cada fila. `data-edit` y `data-delete` contienen el ID del producto. Al pulsar Editar, el listener busca ese producto dentro de la lista cargada y lo pasa a `openProductForm`. Al pulsar Eliminar, pide confirmación y envía `DELETE /api/productos/ID`. Si el producto ya tiene ventas relacionadas, MySQL puede impedir el borrado y la API responde con un conflicto.

renderCategoryTabs construye las pestañas y marca la seleccionada. selectInventoryCategory cambia el valor del selector, redibuja esas pestañas y puede recargar la tabla. selectLowStock marca o desmarca la casilla y puede recargar. renderCategorySummary busca los datos de la categoría seleccionada y escribe su total diario, fecha y cantidad.

Escribir en la búsqueda provoca una nueva consulta; cambiar stock bajo también. La opción refreshSummary=false evita pedir otra vez el resumen cuando solo cambió la búsqueda. Así puedes explicar para qué sirve ese parámetro sin memorizar la llamada completa.

PANTALLA 04 · Interfaz real con datos ilustrativos

The screenshot displays the 'Districarnes' administrative interface. A modal for adding a new product is centered, with fields numbered 1 through 5. Field 1 is 'Nombre', 2 is 'Categoría', 3 is 'Precio por unidad de venta', 4 is 'Existencia disponible', and 5 is 'Producto activo'. A 'Guardar' button is at the bottom. The background shows the 'Inventario' section with a table of products and a sidebar menu.

En el formulario, el número 1 es el nombre; el 2, la categoría; el 3, el precio; el 4, la existencia; y el 5, Guardar. El precio y la existencia dependen de la unidad elegida: si es kg, el precio es por kilogramo y el stock se interpreta en kilogramos. «Activo» controla si podrá ofrecerse en el catálogo de ventas; desactivarlo no borra su información.

BACKEND/src/routes/productRoutes.js: las cuatro operaciones del inventario

CAPTURA 12 · BACKEND/src/routes/productRoutes.js

BACKEND/src/routes/productRoutes.js

CÓDIGO ORIGINAL

```

1  const express = require('express');
2  const productController = require('../controllers/productController');
3  const { requireAuth } = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.use(requireAuth);
8  router.get('/', productController.list);
9  router.post('/', productController.create);
10 router.put('/:id', productController.update);
11 router.delete('/:id', productController.remove);
12
13 module.exports = router;

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Este archivo dice qué controlador atiende cada acción: GET lista, POST crea, PUT /:id actualiza y DELETE /:id elimina. Antes de permitir cualquiera, exige una sesión válida. El texto :id es un espacio variable: /productos/7 significa que el producto solicitado tiene identificador 7.

Aquí no se calcula un precio ni se dibuja una tabla. Si una petición usa el método equivocado o una dirección que no coincide, puede no llegar al controlador esperado. Por eso revisar la ruta es un paso diferente de revisar los campos del formulario.

BACKEND/src/controllers/productController.js: revisar antes de guardar

CAPTURA 21 · BACKEND/src/controllers/productController.js

BACKEND/src/controllers/productController.js

CÓDIGO ORIGINAL

```

1  const productModel = require('../models/productModel');
2
3  function parseProduct(body) {
4    return {
5      nombre: body.nombre?.trim(),
6      tipo_corte: body.tipo_corte?.trim() || null,
7      categoria_id: Number(body.categoria_id),
8      unidad_medida: body.unidad_medida,
9      precio: Number(body.precio),
10     stock: Number(body.stock),
11     activo: body.activo ?? true
12   };
13 }
14
15 function validateProduct(product) {
16   return Boolean(product.nombre)
17     && Number.isInteger(product.categoria_id)
18     && product.categoria_id > 0
19     && ['kg', 'lb', 'unidad'].includes(product.unidad_medida)
20     && Number.isFinite(product.precio)

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

parseProduct toma los datos enviados y los prepara: elimina espacios innecesarios de los textos y transforma ciertas entradas en números. validateProduct comprueba que haya nombre, categoría numérica válida, unidad permitida, precio positivo y stock no negativo. parseld comprueba que un identificador sea un entero positivo.

create utiliza esas comprobaciones y, si todo está bien, pide al Modelo que inserte el producto. update revisa además el ID y solicita actualizar esa fila. remove verifica el ID y solicita eliminarla. list interpreta los filtros de búsqueda y pide los resultados. Estas funciones deciden la respuesta que recibe el navegador, por ejemplo «Producto creado» o «Producto no encontrado».

La validación actual todavía puede mejorar; no debes afirmar que cubre cualquier entrada imaginable. Lo esencial para entenderla es que escribir un dato en un formulario no da autorización para guardarlo sin revisión en el servidor.

BACKEND/src/models/productModel.js: donde se escribe el SQL

CAPTURA 25 · BACKEND/src/models/productModel.js

BACKEND/src/models/productModel.js	CÓDIGO ORIGINAL
1	const pool = require('../config/database');
2	
3	async function findAll({ search = '', categoryId = null, lowStock = false }) {
4	const searchTerm = `%\${search}%`;
5	const [rows] = await pool.execute(
6	`SELECT p.id, p.nombre, p.tipo_corte, p.categoria_id,
7	c.nombre AS categoria, p.precio, p.stock, p.unidad_medida,
8	p.activo, p.imagen_url
9	FROM productos p
10	INNER JOIN categorias c ON c.id = p.categoria_id
11	WHERE (p.nombre LIKE ? OR p.tipo_corte LIKE ?)
12	AND (? IS NULL OR p.categoria_id = ?)
13	AND (? = FALSE OR p.stock <= 10)
14	ORDER BY p.nombre`,
15	[searchTerm, searchTerm, categoryId, categoryId, lowStock]
16	);
17	return rows;
18	}
19	
20	async function create(product) {

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

findAll consulta productos y los relaciona con categorías para devolver también el nombre de la categoría. Agrega las condiciones de búsqueda y stock bajo y ordena por nombre. La palabra LIKE sirve para buscar coincidencias de texto. Los signos ? reciben valores separados de la consulta.

create ejecuta INSERT, que añade un producto. update ejecuta UPDATE, que cambia datos del producto identificado. remove ejecuta DELETE, que intenta borrar la fila. Las funciones de escritura devuelven el nuevo ID o cuántas filas se afectaron; el controlador usa ese resultado para responder.

Este archivo es el lugar adecuado para buscar cómo se guarda un campo. Pero si el problema es que el botón no abre el formulario, todavía no se ha llegado a este archivo: primero debes revisar la Vista.

6. Nueva venta: del producto seleccionado al cobro

PANTALLA 05 · Interfaz real con datos ilustrativos

Districarnes
Panel administrativo

Cerrar sesión

MENÚ PRINCIPAL

- Inicio
- Inventario
- + Nueva venta**
- Historial
- Administradores

PUNTO DE VENTA

Nueva venta

1 Rascar producto
Nombre, corte o categoría

Categoría
Todas las categorías 1-3 de 3 productos

RES
Punta de Anca
\$ 38.000 / kg · Stock 15 kg
2 Agregar

RES
Lomo Fino
\$ 45.000 / kg · Stock 8 kg
Agregar

CERDO
Costilla de Cerdo
\$ 35.000 / kg · Stock 20 kg
Agregar

← Anterior Página 1 de 1 Siguiente →

Detalle

Cliente
Consumidor final

Método de pago
5 Efectivo

Monto recibido
20000

Cambio \$ 4.800

Punta de Anca
\$ 38.000 por kg · disponible 15 kg
3 400 **4** Gramos **Quitar**

Total \$ 15.200
6 Registrar venta

Acceso rápido
Elige un módulo. Solo verás la información que necesitas.

En esta captura, el número 1 identifica la búsqueda del catálogo; el 2, Agregar; el 3, la cantidad; el 4, su unidad; el 5, el método de pago; y el 6, Registrar venta. El carrito está al lado del catálogo en escritorio. Dentro de cada línea también existe Quitar. El cliente y el efectivo recibido se escriben en la zona de cobro.

Agregar no guarda una venta y Quitar no borra una venta registrada. Ambos modifican una compra que aún estás preparando. El cambio definitivo ocurre cuando el servidor acepta Registrar venta. Esto explica por qué debes distinguir la información temporal del carrito de los registros duraderos en MySQL.

FRONTEND/js/sales.js: catálogo y carrito

CAPTURA 36 · FRONTEND/js/sales.js

FRONTEND/js/sales.js

CÓDIGO ORIGINAL

```

1 import { apiRequest } from './api.js';
2 import { $, escapeHtml, formatDate, formatDateTime, formatMoney, formatQuantity, showMessage, today }
  from './ui.js';
3
4 const PAGE_SIZE = 24;
5 const KG_PER_LB = 0.45359237;
6 let catalogProducts = [];
7 let cart = [];
8 let currentPage = 1;
9 let dailyReport = null;
10
11 function categoryImage(category) {
12   const fileName = category.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '');
13   return `/images/${fileName}.png`;
14 }
15
16 function visibleProducts() {
17   const search = $('#sale-search').value.trim().toLocaleLowerCase('es');
18   const categoryId = Number($('#sale-category-filter').value) || null;
19

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

loadSaleCatalog trae los productos del servidor. visibleProducts selecciona los que están activos, tienen stock y coinciden con la búsqueda y categoría elegidas. renderCatalog toma esa selección y dibuja las tarjetas por páginas de 24. Por eso los botones Anterior y Siguiente cambian qué tarjetas ves, pero no hacen una venta.

categoryImage decide qué imagen usar cuando el producto no trae una propia. Convierte el nombre de categoría a un nombre de archivo. Así, Visceras termina relacionado con visceras.png. Si falta la imagen, eso no demuestra que falte el producto en la base.

Cuando pulsas Agregar, el evento busca el producto y lo copia al arreglo cart si todavía no estaba. Un producto por peso empieza con 400 g; uno por unidad, con cantidad 1. En el comportamiento actual, volver a pulsar Agregar sobre el mismo producto no aumenta la cantidad: debes cambiarla en el campo.

renderCart vuelve visible ese arreglo: crea las líneas, campos de cantidad, listas de unidad y botones Quitar. También escribe el total y habilita o deshabilita Registrar venta según cartIsValid. Si el carrito está vacío o tiene una cantidad inválida, no debe quedar listo para registrar.

quantityInBase convierte lo que escribiste a la unidad del producto. cartTotal utiliza esa cantidad convertida para sumar precios. cartIsValid revisa que cada cantidad sea positiva, válida y no supere el stock que se cargó en la pantalla. convertDisplayedQuantity cambia la cantidad mostrada cuando eliges otra unidad, intentando conservar el mismo peso. saleUnitOptions ofrece solo las unidades que corresponden al producto.

Un ejemplo: si el producto se vende por kilogramo, 400 g deben convertirse en 0,4 kg antes de multiplicar por su precio. Si cuesta \$38.000/kg, el subtotal es \$15.200. Multiplicar 400 directamente por 38.000 sería interpretar gramos como kilogramos.

FRONTEND/js/sales.js: pago y registro

updatePayment muestra los campos de efectivo cuando eliges Efectivo y presenta el cambio. Para Tarjeta y Transferencia oculta esos campos y toma el total como recibido. No realiza un cobro bancario.

initializeSales prepara todas las búsquedas, selectores y botones del módulo. Dentro están las funciones sin nombre que responden a Agregar, cambiar cantidad, cambiar unidad, Quitar, cambiar pago y Registrar venta. Puedes encontrarlas buscando el id del botón o el atributo data-* que aparece en la tarjeta.

Al pulsar save-sale, el código envía los IDs de productos, cantidades, unidades y datos de cobro mediante POST /api/ventas. No envía un precio que el servidor deba aceptar a ciegas. Al recibir éxito, vacía el carrito, limpia cliente y recibido, actualiza el catálogo y llama a showReceipt con el ID de la nueva venta.

BACKEND/src/routes/saleRoutes.js: dónde llegan las solicitudes de ventas

CAPTURA 13 · BACKEND/src/routes/saleRoutes.js

BACKEND/src/routes/saleRoutes.js

CÓDIGO ORIGINAL

```

1  const express = require('express');
2  const saleController = require('../controllers/saleController');
3  const { requireAuth } = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.use(requireAuth);
8  router.get('/', saleController.list);
9  router.get('/resumen-categorias', saleController.dailyCategorySummary);
10 router.get('/:id', saleController.detail);
11 router.post('/', saleController.create);
12
13 module.exports = router;
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

POST / se conecta con create para registrar; GET / se conecta con list para historial; GET /resumen-categorias entrega totales por categoría; GET /:id entrega un recibo. Todas llevan /api/ventas como dirección base y exigen sesión.

La ruta fija resumen-categorias está antes de /:id para que el servidor no intente interpretar esa palabra como el número de una venta. Ese orden es algo concreto que el profesor podría cambiar y que necesitas reconocer.

BACKEND/src/controllers/saleController.js: revisar la solicitud de venta

CAPTURA 22 · BACKEND/src/controllers/saleController.js

BACKEND/src/controllers/saleController.js

CÓDIGO ORIGINAL

```

1  const saleModel = require('../models/saleModel');
2
3  async function list(req, res, next) {
4    try {
5      const date = parseDate(req.query.fecha);
6      const categoryId = parseCategoryId(req.query.categoria);
7      if (date === false || categoryId === false) {
8        return res.status(400).json({ message: 'Los filtros de ventas no son válidos' });
9      }
10     res.json(await saleModel.findAll({ date, categoryId }));
11   } catch (error) {
12     next(error);
13   }
14 }
15
16 function parseDate(value) {
17   if (!value) return null;
18   return /^\\d{4}-\\d{2}-\\d{2}$/.test(value) ? value : false;
19 }
20

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

create comprueba que llegue una lista con productos, que cada producto tenga un ID positivo y que las cantidades y unidades sean válidas. También revisa el método de pago. La cuenta que atendió se toma de la sesión reconocida por el servidor; no depende de que el navegador escriba un administrador cualquiera.

list interpreta la fecha y categoría del historial; dailyCategorySummary interpreta la fecha del resumen; detail revisa el ID del recibo. parseDate y parseCategoryId son auxiliares que preparan esos filtros. Una función auxiliar ayuda a otra y evita repetir la misma comprobación.

BACKEND/src/services/measurementService.js: la conversión que usa el servidor

CAPTURA 16 · BACKEND/src/services/measurementService.js

BACKEND/src/services/measurementService.js	CÓDIGO ORIGINAL
1	const KG_PER_LB = 0.45359237;
2	
3	function convertToBaseQuantity(quantity, saleUnit, baseUnit) {
4	if (baseUnit === 'unidad') {
5	return saleUnit === 'unidad' && Number.isInteger(quantity) ? quantity : NaN;
6	}
7	if (!['g', 'kg', 'lb'].includes(saleUnit)) return NaN;
8	
9	const kilograms = saleUnit === 'g'
10	? quantity / 1000
11	: saleUnit === 'lb'
12	? quantity * KG_PER_LB
13	: quantity;
14	const baseQuantity = baseUnit === 'lb' ? kilograms / KG_PER_LB : kilograms;
15	return Number(baseQuantity.toFixed(3));
16	}
17	
18	module.exports = { convertToBaseQuantity };

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

convertToBaseQuantity convierte gramos, kilogramos y libras a la unidad base guardada en el producto. Para artículos por unidad solo acepta cantidades enteras. Si la combinación no tiene sentido, devuelve un valor inválido que el modelo debe rechazar.

El servidor vuelve a hacer la conversión aunque el navegador ya la haya mostrado. La pantalla puede estar desactualizada o alguien puede modificar una petición; el servidor necesita comprobarla por sí mismo. Actualmente hay una diferencia de redondeo entre ambas capas para algunas cantidades, identificada en la revisión anterior.

BACKEND/src/models/saleModel.js: guardar todo como una operación

CAPTURA 26 · BACKEND/src/models/saleModel.js

BACKEND/src/models/saleModel.js

CÓDIGO ORIGINAL

```

79 async function create({ items, clienteNombre, metodoPago, montoRecibido, administradorId }) {
80   const connection = await pool.getConnection();
81
82   try {
83     await connection.beginTransaction();
84     let total = 0;
85     const validatedItems = [];
86
87     for (const item of items) {
88       const [rows] = await connection.execute(
89         `SELECT id, precio, stock, unidad_medida
90          FROM productos
91          WHERE id = ? AND activo = TRUE
92          FOR UPDATE`,
93         [item.producto_id]
94       );
95       const product = rows[0];
96       const requestedQuantity = Number(item.cantidad);
97       const saleUnit = item.unidad_venta || product?.unidad_medida;
98       const quantity = product

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

create reserva una conexión y abre una transacción. Una transacción agrupa cambios que deben confirmarse juntos: la venta, sus detalles, el movimiento de pago y los efectos de los triggers. No sería correcto guardar el pago si falló el detalle o descontar un producto de una venta que no quedó registrada.

Primero consulta cada producto real, comprueba que esté activo, toma su precio y stock y bloquea la fila mientras decide la operación. Convierte la cantidad, comprueba existencias y suma el total. Luego revisa si el efectivo recibido alcanza. Si algo falla, no debe confirmar una venta parcial.

Después inserta la cabecera en ventas. Esa cabecera contiene cliente, fecha, quién atendió, método de pago y total. Inserta una línea en detalle_venta por cada artículo. En ese momento MySQL ejecuta el trigger que descuenta las existencias. También guarda un movimiento de pago. commit confirma; rollback revierte ante error; release devuelve la conexión para que otras operaciones puedan usarla.

El mismo archivo contiene findAll para el historial, findDailyCategorySummary para los resúmenes y findById para un recibo. Son consultas de información guardada: abrir un recibo no debería volver a descontar inventario.

7. Historial, recibo e impresión

PANTALLA 06 · Interfaz real con datos ilustrativos

Districarnes
Panel administrativo

[Cerrar sesión](#)

MENÚ PRINCIPAL

- Inicio
- Inventario
- Nueva venta
- Historial**
- Administradores

REGISTROS

Historial de ventas

Fecha: 06/09/2026
 Categoría: Todas las categorías

Total del día seleccionado: **\$15.200**
 0,4 en unidades base
 1 venta

VENTA	FECHA	CLIENTE	PAGO	PRODUCTOS	TOTAL	ESTADO
#1	6/9/2026, 10:30:00 a. m.	Cliente de ejemplo	efectivo	1	\$ 15.200	completada

Acceso rápido
Elige un módulo. Solo verás la información que necesitas.

5 Imprimir total del día

4 Ver recibo

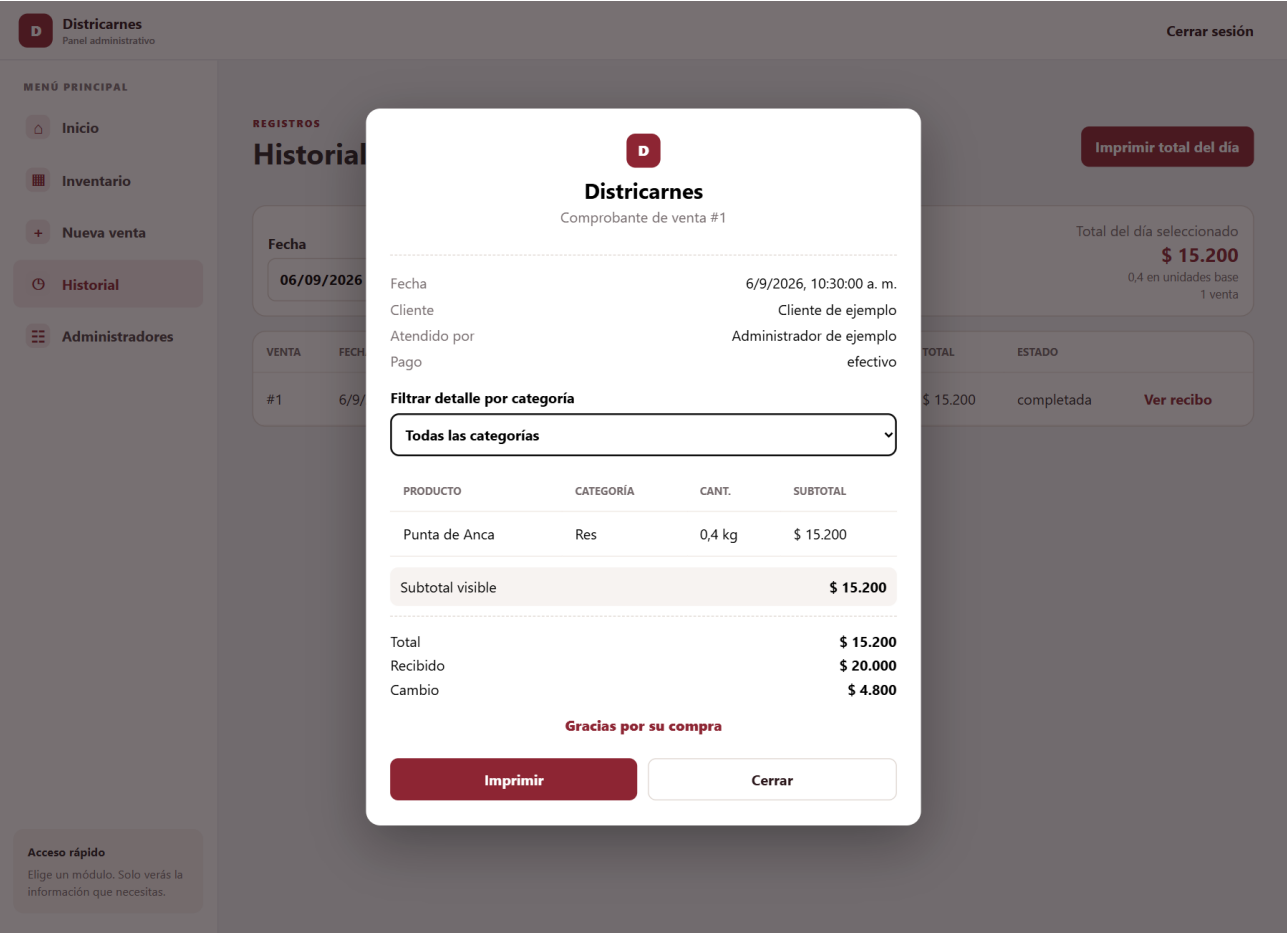
El número 1 elige fecha; el 2, categoría; el 3 muestra el total consultado; el 4 abre el recibo de una fila; y el 5 abre el reporte diario. Una fila representa una venta. Si filtras por categoría, el subtotal mostrado puede ser menor que el total completo de esa venta porque solo considera la categoría elegida.

Las funciones de sales.js que explican esta pantalla

`loadSalesHistory` lee fecha y categoría, pide las ventas y el resumen, dibuja las filas y conserva los datos necesarios para imprimir. Cambiar un filtro vuelve a consultar. El filtro no modifica ni elimina ventas: cambia cuáles ves.

`showReceipt` pide al servidor la venta seleccionada y construye el comprobante. Dentro tiene `renderReceiptItems`, que dibuja los artículos y permite filtrar su detalle por categoría. Ese filtro cambia el subtotal visible, pero conserva el total de la venta completa.

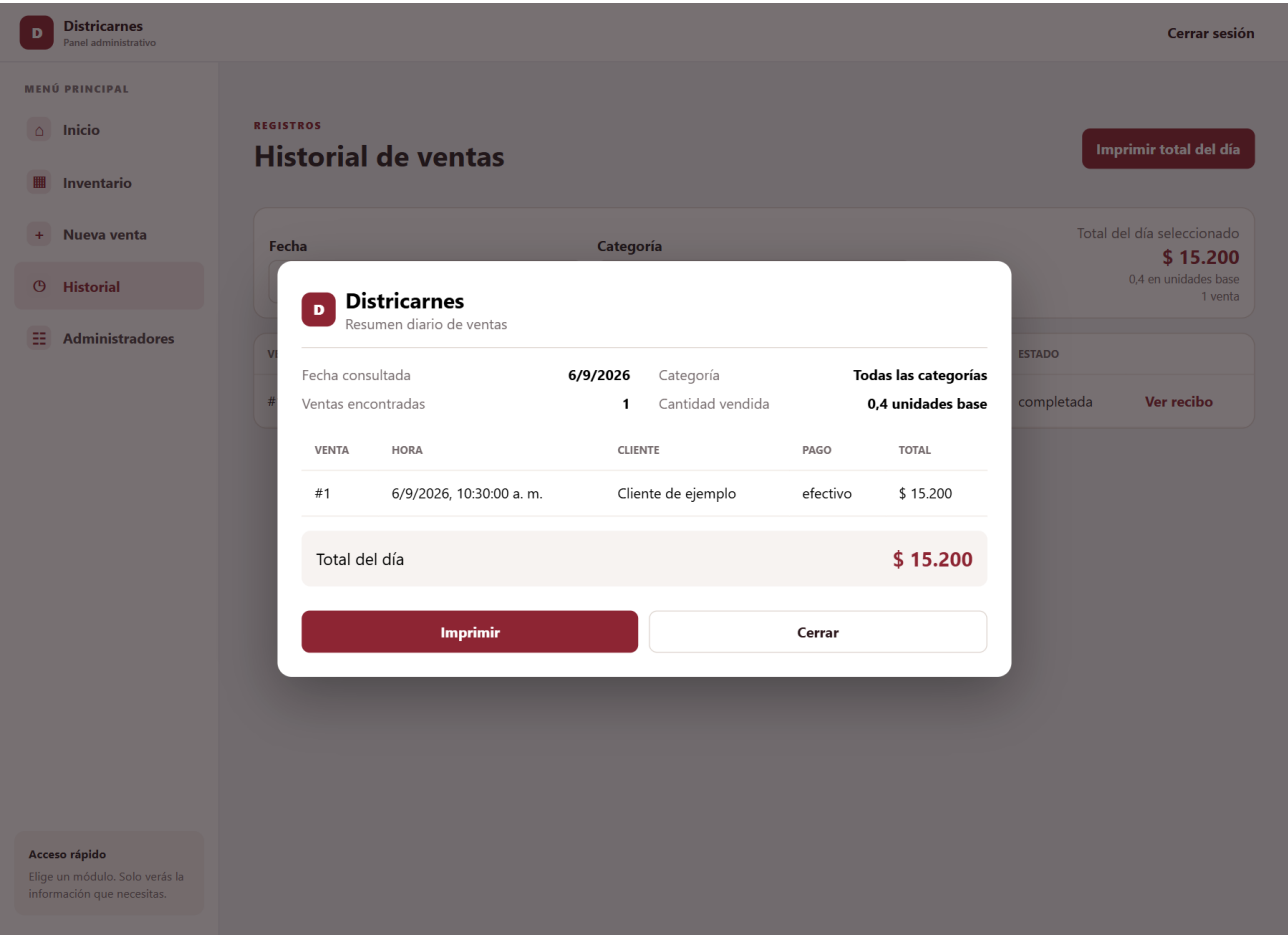
PANTALLA 07 · Interfaz real con datos ilustrativos



En el recibo aparecen número de venta, fecha, cliente, persona que atendió, pago, artículos y totales. Imprimir llama a window.print; Cerrar cierra el diálogo. Los botones se declaran en app.html y sus eventos se conectan en sales.js. El contenido del recibo lo genera showReceipt después de consultar sus datos.

openDailyReport toma los datos que loadSalesHistory dejó preparados y abre otra ventana con el resumen de la fecha y categoría consultadas. No crea otra venta ni una tabla nueva de reportes.

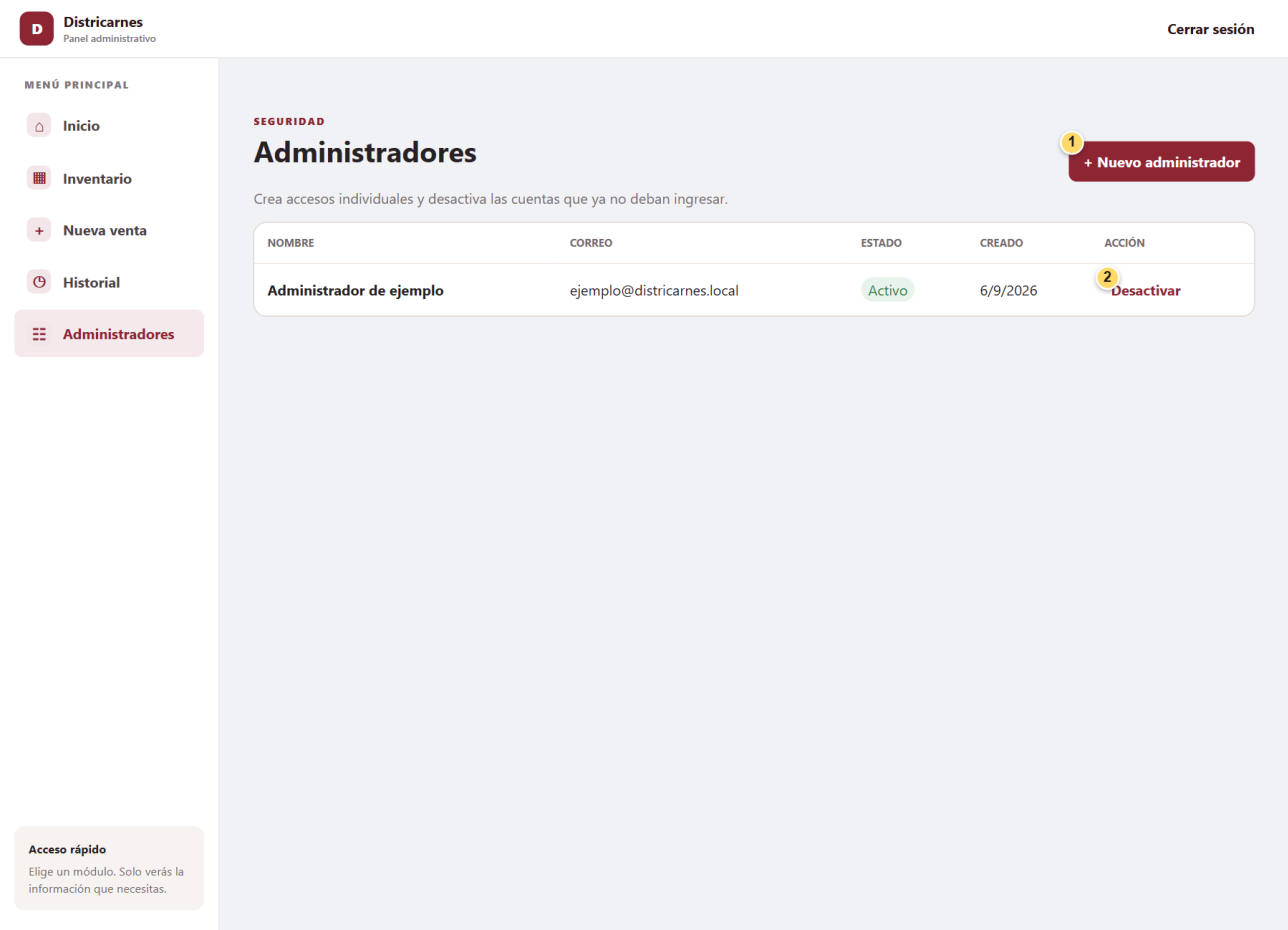
PANTALLA 08 · Interfaz real con datos ilustrativos



Este reporte permite imprimir la lista y el total consultado. Si la información cambia después de cargar el historial, debes volver a consultarlo para que el reporte refleje esos cambios. La impresión utiliza la vista del navegador, no un servicio de generación de comprobantes en el backend.

8. Administradores: quién puede usar el sistema

PANTALLA 09 · Interfaz real con datos ilustrativos



El número 1 abre el formulario para crear una cuenta. El número 2 cambia el estado de la cuenta de esa fila. Activo significa que puede iniciar sesión; Inactivo impide un nuevo ingreso. En esta versión todas las cuentas son administradores: no se encontraron roles separados de cajero y supervisor.

FRONTEND/js/administrators.js: lista, formulario y botones

CAPTURA 35 · FRONTEND/js/administrators.js

FRONTEND/js/administrators.js

CÓDIGO ORIGINAL

```

1 import { apiRequest } from './api.js';
2 import { $, escapeHtml, formatDate, showMessage } from './ui.js';
3
4 function renderAdministrators(administrators) {
5   $('#administrator-rows').innerHTML = administrators.map(administrator => `
6     <tr>
7       <td><strong>${escapeHtml(administrator.nombre)}</strong></td>
8       <td>${escapeHtml(administrator.email)}</td>
9       <td><span class="stock ${administrator.activo ? '' : 'low'}">${administrator.activo ? 'Activo' :
'Inactivo'}</span></td>
10      <td>${formatDate(administrator.fecha_creacion)}</td>
11      <td>
12        <button data-admin-id="${administrator.id}" data-next-active="${administrator.activo ? 'false'
: 'true'}" class="link">
13          ${administrator.activo ? 'Desactivar' : 'Activar'}
14        </button>
15      </td>
16    </tr>
17  `).join('') || '<tr><td colspan="5" class="empty">No hay administradores registrados.</td></tr>';
18 }
19

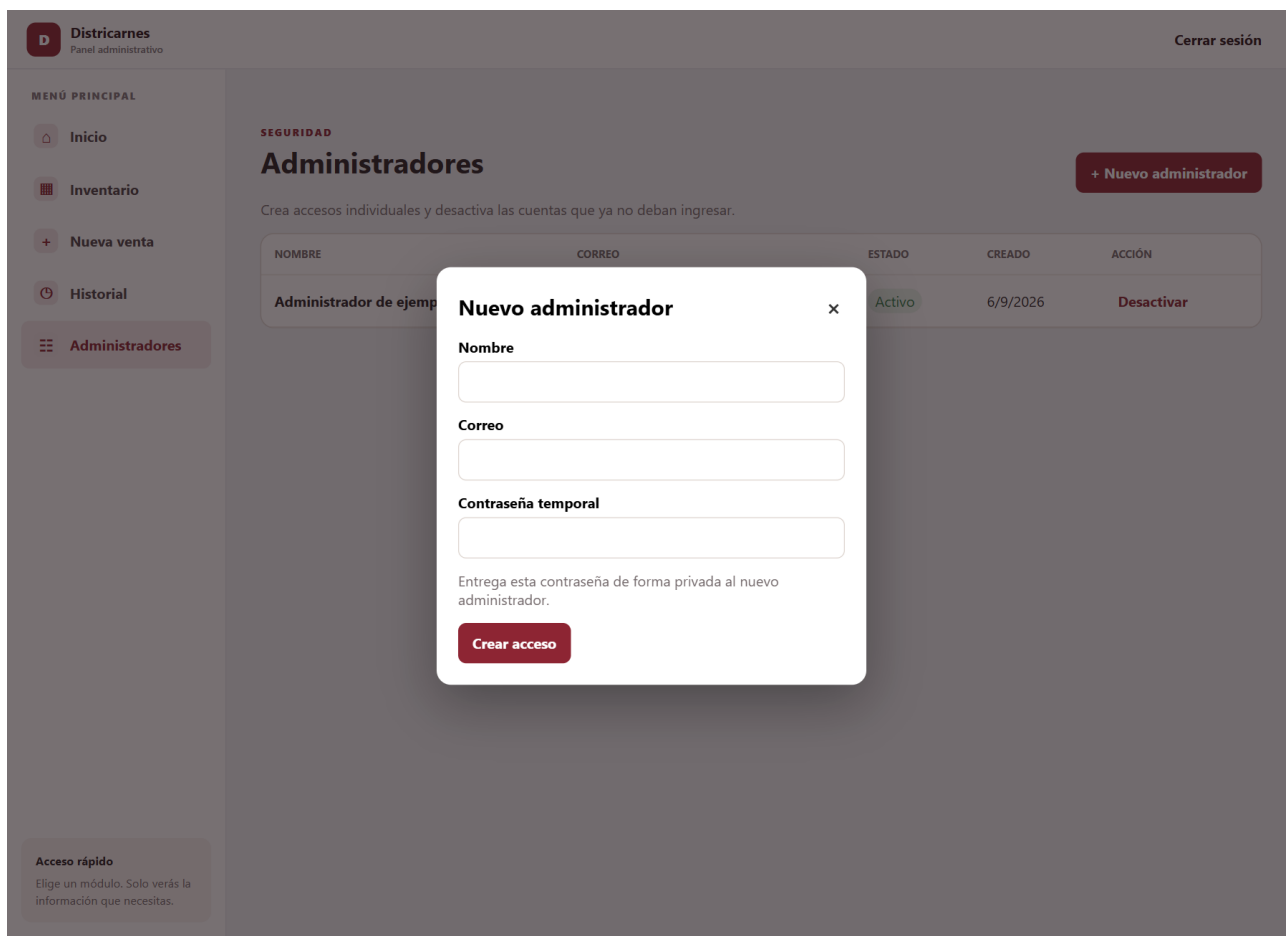
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

loadAdministrators consulta las cuentas. renderAdministrators las convierte en filas y crea el botón Activar o Desactivar según el estado actual. El botón contiene data-admin-id, que identifica la cuenta, y data-next-active, que indica el estado que se quiere asignar.

initializeAdministrators conecta Nuevo administrador, Cerrar, el envío del formulario y los botones de estado. Al enviar el formulario recoge nombre, correo y contraseña y pide POST /api/administradores. Si funciona, limpia campos, cierra y recarga. Para cambiar estado envía PATCH /api/administradores/ID/estado con un verdadero o falso.

PANTALLA 10 · Interfaz real con datos ilustrativos



El formulario está escrito en `app.html`. El botón `Crear acceso` es un botón de envío: por eso la función que debes localizar está en el evento `submit` de `administrator-form`, no necesariamente en un listener de clic con el mismo texto.

BACKEND/src/routes/administratorRoutes.js

CAPTURA 8 · BACKEND/src/routes/administratorRoutes.js

BACKEND/src/routes/administratorRoutes.js	CÓDIGO ORIGINAL
1	<code>const express = require('express');</code>
2	<code>const administratorController = require('../controllers/administratorController');</code>
3	<code>const { requireAuth } = require('../middleware/authMiddleware');</code>
4	
5	<code>const router = express.Router();</code>
6	
7	<code>router.use(requireAuth);</code>
8	<code>router.get('/', administratorController.list);</code>
9	<code>router.post('/', administratorController.create);</code>
10	<code>router.patch('/:id/estado', administratorController.updateStatus);</code>
11	
12	<code>module.exports = router;</code>

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Asocia consultar, crear y cambiar estado con `list`, `create` y `updateStatus` del controlador. Todas requieren sesión. No hay una ruta implementada para cambiar contraseñas desde la pantalla actual; el texto «contraseña temporal» del formulario no demuestra que exista ese flujo posterior.

BACKEND/src/controllers/administratorController.js

CAPTURA 18 · BACKEND/src/controllers/administratorController.js

BACKEND/src/controllers/administratorController.js

CÓDIGO ORIGINAL

```
1 const administratorModel = require('../models/administratorModel');
2
3 async function list(req, res, next) {
4   try {
5     res.json(await administratorModel.findAll());
6   } catch (error) {
7     next(error);
8   }
9 }
10
11 async function create(req, res, next) {
12   try {
13     const nombre = req.body.nombre?.trim();
14     const email = req.body.email?.trim().toLowerCase();
15     const password = req.body.password;
16
17     if (!nombre || !email || !password || password.length < 6) {
18       return res.status(400).json({ message: 'Nombre, correo y una contraseña de 6 caracteres son obligatorios' });
19     }
20   }
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

`list` pide y devuelve cuentas. `create` revisa nombre, correo y longitud mínima de la clave, y solicita guardarla. Si el correo ya existe, responde que está duplicado. `updateStatus` comprueba ID y estado booleano e impide desactivar la misma cuenta que está atendiendo la solicitud.

Esa última restricción protege contra autodesactivarse en ese momento. No soluciona todavía todos los casos de sesión: cuando desactivas otra cuenta, su sesión que ya estaba abierta puede seguir reconocida. Conviene conocer esta limitación y no prometer que la desconexión sea inmediata.

BACKEND/src/models/administratorModel.js

CAPTURA 23 · BACKEND/src/models/administratorModel.js

BACKEND/src/models/administratorModel.js

CÓDIGO ORIGINAL

```

1  const pool = require('../config/database');
2
3  async function findActiveByCredentials(email, password) {
4    const [rows] = await pool.execute(
5      `SELECT id, nombre, email
6        FROM administradores
7        WHERE email = ?
8          AND password_hash = SHA2(?, 256)
9          AND activo = TRUE`,
10     [email, password]
11   );
12   return rows[0] || null;
13 }
14
15 async function findAll() {
16   const [rows] = await pool.query(
17     `SELECT id, nombre, email, activo, fecha_creacion
18       FROM administradores
19       ORDER BY activo DESC, nombre`
20   );

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

findActiveByCredentials busca una cuenta activa con el correo y hash de contraseña adecuados; la usa el login. findAll consulta los datos que se muestran en Administradores sin devolver el hash. create inserta la cuenta y transforma la clave con SHA2. setStatus cambia activo en la cuenta indicada.

Este archivo se relaciona tanto con acceso como con mantenimiento de cuentas. Por eso no todos los modelos corresponden a una única pantalla. El modo de guardar contraseñas necesita mejorarse, pero debes explicar primero lo que el código actual realiza.

9. Las categorías también recorren varios archivos

BACKEND/src/routes/categoryRoutes.js

CAPTURA 10 · BACKEND/src/routes/categoryRoutes.js

BACKEND/src/routes/categoryRoutes.js

CÓDIGO ORIGINAL

```

1  const express = require('express');
2  const categoryController = require('../controllers/categoryController');
3  const { requireAuth } = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.get('/', requireAuth, categoryController.list);
8
9  module.exports = router;

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Esta ruta permite GET /api/categorias con sesión válida. Se utiliza al preparar los filtros del panel. No existen rutas de crear, editar o eliminar categorías en la aplicación revisada.

BACKEND/src/controllers/categoryController.js

CAPTURA 20 · BACKEND/src/controllers/categoryController.js

BACKEND/src/controllers/categoryController.js

CÓDIGO ORIGINAL

```
1 const categoryModel = require('../models/categoryModel');
2
3 async function list(req, res, next) {
4   try {
5     res.json(await categoryModel.findAll());
6   } catch (error) {
7     next(error);
8   }
9 }
10
11 module.exports = { list };
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Su función list pide al modelo todas las categorías y responde con ellas. Si la consulta falla, entrega el error al manejador compartido. Es breve porque no tiene formularios de escritura ni filtros complejos que interpretar.

BACKEND/src/models/categoryModel.js

CAPTURA 24 · BACKEND/src/models/categoryModel.js

BACKEND/src/models/categoryModel.js

CÓDIGO ORIGINAL

```
1 const pool = require('../config/database');
2
3 async function findAll() {
4   const [rows] = await pool.query(
5     'SELECT id, nombre FROM categorias ORDER BY nombre'
6   );
7   return rows;
8 }
9
10 module.exports = { findAll };
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

findAll consulta ID y nombre de categorías y ordena por nombre. El ID es el que se envía al filtrar; el nombre es lo que ve la persona. No conviene usar el texto visible como sustituto del ID porque los nombres pueden cambiar.

app.js del frontend recibe esa lista y se la pasa a inventory.js y sales.js. Ambos llenan opciones y pestañas con ella. Una falla en categorías al inicializar puede afectar varios módulos, aunque la persona crea que solo falló un selector.

10. Los archivos que conectan todo

FRONTEND/js/api.js: el encargado de llevar y traer solicitudes

CAPTURA 29 · FRONTEND/js/api.js

FRONTEND/js/api.js

CÓDIGO ORIGINAL

```
1 const TOKEN_KEY = 'districarnes_session';
2
3 export function hasSession() {
4   return Boolean(sessionStorage.getItem(TOKEN_KEY));
5 }
6
7 export async function apiRequest(path, { method = 'GET', body } = {}) {
8   const token = sessionStorage.getItem(TOKEN_KEY);
9   const headers = { Accept: 'application/json' };
10
11   if (token) headers.Authorization = `Bearer ${token}`;
12   if (body !== undefined) headers['Content-Type'] = 'application/json';
13
14   const response = await fetch(`/api${path}`, {
15     method,
16     headers,
17     body: body === undefined ? undefined : JSON.stringify(body)
18   });
19   const data = await response.json().catch(() => ({}));
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

`apiRequest` es la función que usan las pantallas para hablar con el servidor. Recibe una dirección, un método y, cuando corresponde, datos. Agrega `/api` a la dirección, incluye el token de sesión, convierte los datos a JSON y espera la respuesta. Si la operación fue rechazada, lanza un error para que la pantalla lo muestre.

`hasSession` solo comprueba si el navegador tiene un token guardado. `startSession` llama al login y guarda el token devuelto. `closeSession` pide cerrar el acceso y limpia la copia local incluso si hubo un error de comunicación. Guardar un token local no garantiza que el servidor todavía lo reconozca.

La ventaja de `apiRequest` es que no repites la preparación de solicitudes en cada pantalla. Si todas dejan de enviar el token, revisar este archivo puede ser más útil que revisar cada controlador por separado.

FRONTEND/js/ui.js: pequeñas ayudas de presentación

CAPTURA 30 · FRONTEND/js/ui.js

FRONTEND/js/ui.js

CÓDIGO ORIGINAL

```

1 export const $ = selector => document.querySelector(selector);
2
3 export const formatMoney = value => new Intl.NumberFormat('es-CO', {
4   style: 'currency',
5   currency: 'COP',
6   maximumFractionDigits: 0
7 }).format(Number(value));
8
9 export const formatDateTime = value => new Date(value).toLocaleString('es-CO');
10 export const formatDate = value => {
11   const normalizedValue = /^\\d{4}-\\d{2}-\\d{2}$/.test(value) ? `${value}T00:00:00` : value;
12   return new Date(normalizedValue).toLocaleDateString('es-CO');
13 };
14
15 export function today() {
16   const date = new Date();
17   const offset = date.getTimezoneOffset();
18   return new Date(date.getTime() - offset * 60000).toISOString().slice(0, 10);
19 }

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

La función `$` abrevia la búsqueda de un elemento HTML por selector. `formatMoney` presenta valores como moneda colombiana; no consulta precios. `formatQuantity` presenta cantidades con su unidad. `formatDate` y `formatDateTime` presentan fechas; `today` obtiene la fecha del navegador para filtros.

`escapeHtml` permite incluir un texto dentro del HTML sin interpretar sus signos como etiquetas. Por ejemplo, un nombre con `<` y `>` debería seguir siendo un nombre visible, no convertirse en elementos de página. `showMessage` escribe un aviso de éxito o error y lo quita después de unos segundos; `messageTimer` evita que un temporizador anterior quite un aviso nuevo demasiado pronto.

Estas funciones se reutilizan en varios módulos. No registran ventas ni modifican productos. Si todos los precios se ven con un formato incorrecto, `formatMoney` es una referencia directa; si el valor guardado es incorrecto, el problema puede estar en el cálculo y no en el formato.

BACKEND/src/middleware/authMiddleware.js: reconocer la sesión

CAPTURA 14 · BACKEND/src/middleware/authMiddleware.js

BACKEND/src/middleware/authMiddleware.js

CÓDIGO ORIGINAL

```
1 const sessionService = require('../services/sessionService');
2
3 function requireAuth(req, res, next) {
4   const authorization = req.headers.authorization || '';
5   const token = authorization.startsWith('Bearer ')
6     ? authorization.slice(7)
7     : null;
8
9   const user = token ? sessionService.find(token) : null;
10  if (!user) {
11    return res.status(401).json({ message: 'Sesión no válida' });
12  }
13
14  req.token = token;
15  req.user = user;
16  next();
17 }
18
19 module.exports = { requireAuth };
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

`requireAuth` mira el token enviado en la cabecera `Authorization` y pregunta a `sessionService` si existe. Si no existe, detiene la petición con 401. Si existe, coloca los datos de la persona en `req.user` y permite continuar al controlador.

Ocultar el menú no reemplaza esta comprobación. Una solicitud podría hacerse sin pulsar un botón, por lo que el servidor debe proteger las operaciones de datos. `requireAuth` no vuelve a pedir la contraseña: reconoce una sesión creada antes.

BACKEND/src/services/sessionService.js: recordar quién entró

CAPTURA 17 · BACKEND/src/services/sessionService.js

BACKEND/src/services/sessionService.js

CÓDIGO ORIGINAL

```

1  const crypto = require('crypto');
2
3  const sessions = new Map();
4
5  function create(administrator) {
6    const token = crypto.randomBytes(24).toString('hex');
7    sessions.set(token, {
8      id: administrator.id,
9      nombre: administrator.nombre,
10     email: administrator.email
11   });
12   return token;
13 }
14
15 function find(token) {
16   return sessions.get(token) || null;
17 }
18
19 function remove(token) {
20   sessions.delete(token);

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

create genera un token aleatorio y guarda una relación entre ese token y los datos básicos de la cuenta. find busca esa relación. remove la elimina al cerrar sesión. Todo se conserva en un Map de la memoria de Node, no en una tabla persistente.

Por eso reiniciar Node hace que los tokens anteriores dejen de estar reconocidos. Los productos y ventas no se pierden por ese reinicio porque viven en MySQL. Esta diferencia entre información temporal y persistente explica un comportamiento que puede aparecer durante el examen.

BACKEND/src/middleware/errorMiddleware.js: cómo se informan fallos

CAPTURA 15 · BACKEND/src/middleware/errorMiddleware.js

BACKEND/src/middleware/errorMiddleware.js

CÓDIGO ORIGINAL

```

1  function notFound(req, res) {
2    res.status(404).json({ message: `Ruta no encontrada: ${req.method} ${req.originalUrl}` });
3  }
4
5  function errorHandler(error, _req, res, _next) {
6    const databaseStatus = error.code === 'ER_ROW_IS_REFERENCED_2' ? 409 : null;
7    const status = error.status || databaseStatus || 500;
8    if (status === 500) console.error(error);
9    const message = status === 500
10     ? 'Ocurrió un error interno en el servidor'
11     : error.message;
12    res.status(status).json({ message });
13  }
14
15  module.exports = { notFound, errorHandler };

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

notFound responde cuando ninguna ruta atiende la petición. errorHandler recibe errores que los controladores le pasan y construye una respuesta. Algunos conflictos se convierten a 409; los errores internos se registran en consola y se muestran con un mensaje general.

Si ves 404, puede faltar una ruta o no existir el recurso pedido. Si ves 401, revisa la sesión. Si ves 400, revisa los datos enviados. Un 500 requiere mirar el servidor: el mensaje general de pantalla no explica por sí solo la causa.

11. ¿Qué pasa al encender el proyecto?

INICIAR.bat

CAPTURA 2 · INICIAR.bat

INICIAR.bat

CÓDIGO ORIGINAL

```
1 @echo off
2 title Districarnes
3 cd /d "%~dp0BACKEND"
4 if not exist node_modules (
5   echo Instalando dependencias...
6   call npm install
7 )
8 if not exist .env (
9   echo Falta configurar BACKEND\.env
10  echo Crea el archivo .env con los datos indicados en README.md.
11  pause
12  exit /b 1
13 )
14 start "" http://localhost:3000
15 call npm start
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Este archivo facilita iniciar el proyecto en Windows. Entra a BACKEND, instala dependencias si falta node_modules, comprueba que exista .env, abre el navegador y ejecuta npm start. No construye la base de datos ni reemplaza MySQL Server.

BACKEND/package.json y BACKEND/package-lock.json

CAPTURA 3 · BACKEND/package.json

BACKEND/package.json

CÓDIGO ORIGINAL

```

1 {
2   "name": "districarnes-backend",
3   "version": "1.0.0",
4   "private": true,
5   "description": "API MVC para el sistema POS Disticarnes",
6   "main": "src/server.js",
7   "scripts": {
8     "start": "node src/server.js",
9     "dev": "node --watch src/server.js"
10  },
11  "dependencies": {
12    "dotenv": "^16.4.7",
13    "express": "^4.21.2",
14    "mysql2": "^3.12.0"
15  }
16 }
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

CAPTURA 4 · BACKEND/package-lock.json

BACKEND/package-lock.json

CÓDIGO ORIGINAL

```

1 {
2   "name": "districarnes-backend",
3   "version": "1.0.0",
4   "lockfileVersion": 3,
5   "requires": true,
6   "packages": {
7     "": {
8       "name": "districarnes-backend",
9       "version": "1.0.0",
10      "dependencies": {
11        "dotenv": "^16.4.7",
12        "express": "^4.21.2",
13        "mysql2": "^3.12.0"
14      }
15    },
16    "node_modules/@types/node": {
17      "version": "26.1.1",
18      "resolved": "https://registry.npmjs.org/@types/node/-/node-26.1.1.tgz",
19      "integrity": "sha512-nxAKRSVkn1Y0JC1W8ky/fTfkGsMmcrRsbx+3XoZE+rMOX71kLYTV7fLXpqud1GbbpP5TuffXfQfX7fH2GgZREw==",
20      "peer": true,
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

package.json declara qué necesita el backend y qué comandos se pueden ejecutar. npm start corresponde a node src/server.js. npm run dev utiliza el mismo archivo con vigilancia de cambios. Express atiende HTTP, mysql2 permite consultar MySQL y dotenv carga variables locales.

package-lock.json registra versiones concretas de dependencias para instalar de forma consistente. No contiene las reglas de inventario. Si quieres cambiar el descuento de stock, no lo buscas aquí; si quieres saber qué se ejecuta al arrancar, miras scripts en package.json.

BACKEND/src/server.js

CAPTURA 5 · BACKEND/src/server.js

BACKEND/src/server.js

CÓDIGO ORIGINAL

```

1  require('dotenv').config();
2
3  const app = require('./app');
4  const database = require('./config/database');
5
6  const port = Number(process.env.PORT) || 3000;
7
8  function handleServerError(error) {
9    const message = error.code === 'EADDRINUSE'
10     ? `El puerto ${port} ya está ocupado. Cierra el servidor anterior antes de iniciar otro.`
11     : `No fue posible iniciar el servidor: ${error.message}`;
12    console.error(message);
13    process.exit(1);
14  }
15
16  async function startServer() {
17    try {
18      const connection = await database.checkConnection();
19      console.log(
20        `MySQL conectado: base ${connection.database_name} (MySQL ${connection.mysql_version})`

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

startServer comprueba primero la conexión con la base y luego abre el puerto para atender al navegador. handleServerError informa, por ejemplo, si el puerto ya está ocupado. Este archivo decide cuándo empezar a escuchar; app.js define qué atender una vez que el servidor está escuchando.

BACKEND/src/app.js

CAPTURA 6 · BACKEND/src/app.js

BACKEND/src/app.js

CÓDIGO ORIGINAL

```

1  const path = require('path');
2  const express = require('express');
3
4  const apiRoutes = require('./routes');
5  const { notFound, errorHandler } = require('./middleware/errorMiddleware');
6
7  const app = express();
8  const frontendPath = path.join(__dirname, '..', '..', 'FRONTEND');
9
10 app.use(express.json());
11 app.use(express.static(frontendPath, { index: false }));
12
13 app.use('/api', apiRoutes);
14
15 app.get('/', (_req, res) => res.sendFile(path.join(frontendPath, 'index.html')));
16 app.get('/app', (_req, res) => res.sendFile(path.join(frontendPath, 'app.html')));
17
18 app.use(notFound);
19 app.use(errorHandler);
20

```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Prepara Express para recibir JSON, entregar los archivos de FRONTEND y montar las rutas /api. También indica que / entrega index.html y /app entrega app.html. Al final conecta los manejadores de error.

El app.js del backend y el app.js del frontend tienen el mismo nombre, pero tareas diferentes. El primero organiza el servidor; el segundo organiza el panel dentro del navegador. Debes fijarte en la carpeta antes de explicar un archivo.

BACKEND/src/routes/index.js

CAPTURA 11 · BACKEND/src/routes/index.js

BACKEND/src/routes/index.js

CÓDIGO ORIGINAL

```
1  const express = require('express');
2
3  const authRoutes = require('./authRoutes');
4  const categoryRoutes = require('./categoryRoutes');
5  const productRoutes = require('./productRoutes');
6  const saleRoutes = require('./saleRoutes');
7  const administratorRoutes = require('./administratorRoutes');
8
9  const router = express.Router();
10
11 router.use(authRoutes);
12 router.use('/categorias', categoryRoutes);
13 router.use('/productos', productRoutes);
14 router.use('/ventas', saleRoutes);
15 router.use('/administradores', administratorRoutes);
16
17 module.exports = router;
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Reúne las rutas de autenticación, categorías, productos, ventas y administradores. Puedes entenderlo como el punto donde se organizan las direcciones del sistema. El archivo de productos escribe sus rutas desde /; este agrupador añade /productos y app.js añade /api. Entre los tres forman la dirección final.

BACKEND/src/config/database.js y BACKEND/.env

CAPTURA 7 · BACKEND/src/config/database.js

BACKEND/src/config/database.js

CÓDIGO ORIGINAL

```
1 const mysql = require('mysql2/promise');
2
3 const pool = mysql.createPool({
4   host: process.env.DB_HOST || 'localhost',
5   port: Number(process.env.DB_PORT) || 3306,
6   user: process.env.DB_USER || 'root',
7   password: process.env.DB_PASSWORD || '',
8   database: process.env.DB_NAME || 'districarnes',
9   waitForConnections: true,
10  connectionLimit: 10
11 });
12
13 async function checkConnection() {
14   const [rows] = await pool.query(
15     'SELECT DATABASE() AS database_name, VERSION() AS mysql_version'
16   );
17   return rows[0];
18 }
19
20 pool.checkConnection = checkConnection;
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

database.js prepara un pool, es decir, conexiones reutilizables con MySQL. Los modelos lo importan para ejecutar sus consultas. checkConnection pregunta qué base está conectada y qué versión de MySQL responde; server.js usa esa comprobación antes de arrancar.

.env contiene la configuración del equipo: dirección y puerto de MySQL, usuario, contraseña, nombre de base y puerto de la aplicación. Cambiar el nombre de base selecciona otra; no crea automáticamente sus tablas. Las claves privadas no deben aparecer en capturas del manual ni entregarse como documentación pública.

12. Dónde está la apariencia y por qué se mueven los botones

FRONTEND/css/styles.css

CAPTURA 37 · FRONTEND/css/styles.css

FRONTEND/css/styles.css	CÓDIGO ORIGINAL
1	:root {
2	font-family:Inter,system-ui,sans-serif;
3	color:#272124;
4	background:#f5f2ee;
5	--primary:#8d2533;
6	--dark:#2b171b;
7	--border:#ddd4ce
8	}
9	
10	* {
11	box-sizing:border-box
12	}
13	
14	body {
15	margin:0
16	}
17	
18	button,input,select {
19	font:inherit
20	}

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Este archivo organiza colores, tamaños, espacios, columnas, tablas, formularios y ventanas. Las reglas que comienzan con punto se aplican a clases HTML, como `.card` o `.hidden`. El JavaScript puede cambiar una clase y el CSS cambia entonces la apariencia correspondiente.

`.hidden` oculta secciones del panel; `.active` distingue una pestaña o botón seleccionado; `.stock.low` destaca una existencia baja. `.sales-layout` coloca catálogo y carrito; `.side-nav` organiza el menú; los estilos de dialog dan forma a formularios emergentes. Los nombres ayudan a relacionar lo que ves con el bloque de apariencia que lo controla.

Las reglas `@media` adaptan el diseño al ancho disponible. En escritorio el menú está a un lado y el carrito junto al catálogo; en pantallas pequeñas el contenido se reorganiza. Por eso «dónde está el botón» debe interpretarse según el dispositivo, aunque el ID y la función que lo atienden no cambien.

`@media print` decide qué queda visible al imprimir. Las acciones Imprimir llaman a `window.print`, y estas reglas ocultan el panel y muestran el diálogo abierto. Si el recibo sale con menú y botones, debes revisar este bloque además del evento de impresión.

FRONTEND/images: res.png, cerdo.png, pollo.png, pescado.png y visceras.png

Son las imágenes que `sales.js` puede usar para las categorías del catálogo. No contienen funciones. Su relación con el programa es el nombre y la dirección del archivo: `categoryImage` los calcula y Express los entrega como recursos de FRONTEND.

13. La base de datos explicada como información del negocio

BACKEND/database/schema.sql

CAPTURA 38 · BACKEND/database/schema.sql

BACKEND/database/schema.sql

CÓDIGO ORIGINAL

```

32 CREATE TABLE productos (
33   id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
34   nombre VARCHAR(100) NOT NULL,
35   tipo_corte VARCHAR(100),
36   categoria_id INT UNSIGNED NOT NULL,
37   imagen_url VARCHAR(255),
38   unidad_medida ENUM('kg', 'lb', 'unidad') NOT NULL DEFAULT 'kg',
39   precio DECIMAL(10,2) NOT NULL,
40   stock DECIMAL(10,3) NOT NULL DEFAULT 0,
41   activo BOOLEAN NOT NULL DEFAULT TRUE,
42   fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
43   fecha_actualizacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
44   ON UPDATE CURRENT_TIMESTAMP,
45   CONSTRAINT fk_producto_categoria
46   FOREIGN KEY (categoria_id) REFERENCES categorias(id)
47   ON UPDATE CASCADE ON DELETE RESTRICT
48 );

```

Modelo de datos: precio y stock se interpretan usando unidad_medida; categoria_id relaciona el producto con categorías.

Este archivo contiene instrucciones para construir tablas, relaciones, datos iniciales y triggers. La base que está funcionando vive en MySQL Server. El archivo no se ejecuta cada vez que pulsas un botón: las consultas de los modelos trabajan con los objetos ya instalados.

administradores guarda las cuentas; categorias clasifica artículos; productos guarda el catálogo, su precio y existencias. ventas guarda la información general de cada compra; detalle_venta guarda los productos vendidos en esa compra. movimientos registra importes asociados y auditoria_productos conserva cambios de productos.

La separación entre ventas y detalle_venta es fácil de entender con un recibo: arriba aparecen fecha, cliente, pago y total; debajo puede haber varios artículos. Guardar toda la cabecera repetida por cada artículo no es la organización usada aquí. Una venta se relaciona con varias líneas de detalle.

Las relaciones protegen la coherencia. Un producto necesita una categoría existente. Una línea de venta necesita un producto y una venta existentes. Un producto que ya aparece en ventas no se borra libremente, porque perderías la referencia de lo vendido. Desactivar y eliminar son acciones distintas.

Un trigger es una regla que MySQL ejecuta automáticamente ante un cambio.

trg_producto_validar_insert y trg_producto_validar_update impiden precio no positivo y stock negativo.

trg_producto_auditar_update y trg_producto_auditar_delete registran cambios y borrados.

trg_administrador_validar_email comprueba una forma básica de correo al insertar cuentas.

trg_detalle_validar_insert revisa cantidad, precio y disponibilidad al insertar un detalle.

trg_detalle_descontar_stock descuenta su cantidad del producto después de insertar. Por eso, si

preguntas «¿qué instrucción baja el inventario cuando vendo?», la respuesta termina en ese trigger. El UPDATE que hace también provoca la auditoría de productos.

El subtotal de cada detalle se calcula automáticamente como cantidad por precio_unitario. La aplicación conserva ese precio de venta, aunque después cambie el precio del catálogo. Sin embargo, los recibos actuales consultan algunos otros datos del producto desde el catálogo vigente; esa conservación histórica aún puede mejorarse.

El inicio del script elimina tablas antes de reconstruirlas. No debes ejecutarlo entero para reparar una función de pantalla ni actualizar una instalación que debe conservar sus ventas. Cambiar el texto del trigger en el archivo tampoco cambia el trigger que ya existe en MySQL: instalación de código SQL y uso del sistema son momentos distintos.

14. Los demás archivos: documentación y herramientas

README.md explica instalación y uso general. INFORME_ESTRUCTURA_CODIGO.md documenta una revisión anterior. GUIA_REVISION_Y_EXAMEN.md contiene el mapa de operaciones y prácticas. ARQUITECTURA_ARCHIVO_POR_ARCHIVO.md es la referencia técnica detallada que acompaña esta explicación. Ninguno atiende una petición de venta.

.gitignore indica archivos que no deben agregarse a Git, como configuración privada y dependencias. No borra archivos ni protege con contraseña. node_modules contiene librerías instaladas y .npm-cache es caché de npm: no debes cambiar sus archivos para reparar una regla de negocio propia.

build_integracion_tests.mjs crea un Excel con un plan de pruebas. Organiza casos, aplica formato, agrega estados y guarda el documento y su vista previa. Que se genere el Excel no significa que se hayan ejecutado sus pruebas. outputs/integracion_districarnes contiene esos resultados documentales.

Los scripts manual_visual.py, capturar_manual.cjs y capturar_recorrido.cjs, junto con este documento, son herramientas añadidas para preparar los manuales. No forman parte del flujo de ventas. Las capturas y PDF están bajo output/manual_visual. La versión explicada se prepara además con generar_manual_explicado.py.

15. Un ejemplo completo para explicárselo al profesor

«Mi proyecto administra el inventario y las ventas de una carnicería. La persona entra con una cuenta, consulta productos por categoría y prepara una venta. La Vista presenta controles y envía solicitudes a una API. Las rutas seleccionan el Controlador, el Controlador revisa los datos y el Modelo ejecuta las operaciones en MySQL».

«Si creo un producto, el botón abre un formulario. Guardar reúne los campos y manda POST /api/productos. El controlador valida y el modelo inserta. Después la Vista consulta otra vez y muestra la nueva fila. Si edito, el mismo formulario conserva el ID y manda PUT para modificar ese producto».

«Si vendo 400 gramos a \$38.000 por kilogramo, se convierten a 0,4 kg y el total es \$15.200. El servidor consulta precio y stock reales, guarda una venta y sus detalles dentro de una transacción, y un trigger

descuenta las existencias. Con \$20.000 recibidos quedan \$4.800 de cambio. El recibo consulta lo que se guardó».

Esa explicación une lo que hace el negocio con lo que hace el código. En el examen, comienza por la acción visible y sigue la cadena. Si no aparece el formulario, revisa Vista y evento. Si la solicitud llega pero se rechaza, revisa datos y Controlador. Si la operación devuelve resultados incorrectos, revisa Modelo, consultas y reglas de MySQL.

16. Referencia detallada de funciones y bloques

La parte siguiente conserva la descripción técnica por archivo para consultar nombres, campos y condiciones exactas. No necesitas memorizarla seguida. Úsala después del recorrido anterior: localiza el archivo que ya entendiste y compara la explicación con sus funciones y capturas de código.

Referencia 1. Archivos de raíz y configuración

README.md

Es el manual de entrada: tecnologías, estructura, endpoints, base e instalación. No se ejecuta. Debe describir lo que realmente está implementado. Su vínculo con el código es documental: si cambia una ruta, un comando o la ubicación del schema, se actualiza. No usar su listado como sustituto de verificar las funciones.

INFORME_ESTRUCTURA_CODIGO.md

Es una revisión anterior, con explicación de módulos y hallazgos. No configura Express ni MySQL. Sus valoraciones son opiniones del informe, no notas del instructor. Puede contener datos desactualizados: el código es la referencia para esta explicación.

GUIA_REVISION_Y_EXAMEN.md

La guía creada en la revisión previa relaciona operaciones, problemas y ejercicios de restauración. Complementa este manual: allí se estudia por funcionalidad; aquí, por archivo y dependencia.

ARQUITECTURA_ARCHIVO_POR_ARCHIVO.md

Este documento. Es material de estudio y no interviene en el funcionamiento.

.gitignore

CAPTURA 1 · .gitignore

.gitignore

CÓDIGO ORIGINAL

```
1 BACKEND/node_modules/
2 BACKEND/.npm-cache/
3 BACKEND/.env
4 *.log
```

Lee las líneas de arriba junto con la explicación del archivo. Los números corresponden al código original.

Indica qué archivos no incorporar como nuevos archivos a Git. Actualmente excluye BACKEND/node_modules, BACKEND/.npm-cache, BACKEND/.env y archivos .log. No borra archivos, no cifra secretos y no deja de rastrear automáticamente uno que ya estuviera versionado. Falta cubrir el node_modules de raíz. Esta copia no mostró carpeta .git en la revisión inicial; tener .gitignore no prueba que ya exista historial de versiones.

INICIAR.bat

@echo off evita mostrar cada instrucción; title cambia el título; cd /d "%~dp0BACKEND" entra al backend respecto a la ubicación del propio archivo, incluso si cambia de unidad. Si falta node_modules ejecuta npm install. Si falta .env, informa y termina con exit /b 1. start abre el navegador y call npm start arranca Node.

Se relaciona con package.json y .env. No instala MySQL ni ejecuta schema.sql. Abre localhost:3000 antes de comprobar que la API arrancó y no lee PORT para construir la URL.

BACKEND/package.json

name y version identifican el paquete. private evita publicación accidental en npm. description describe el propósito. main identifica el punto principal del paquete, pero npm start usa específicamente scripts.start. start ejecuta node src/server.js; dev hace lo mismo con --watch para reiniciar al cambiar archivos. No existe script test.

dependencies declara dotenv (variables de entorno), express (HTTP, rutas y archivos estáticos) y mysql2 (cliente de MySQL). El ^ expresa un rango permitido, no la versión exacta instalada. Ninguna de estas tres dependencias es MySQL Server.

BACKEND/package-lock.json

Archivo generado por npm que registra versiones concretas y árbol de dependencias. lockfileVersion describe el formato. packages contiene entradas por ubicación; version fija versión, resolved identifica procedencia e integrity verifica integridad del paquete. dependencies y engines describen dependencias y requisitos de cada paquete. No se estudia como lógica de ventas ni se modifica manualmente para cambiar un requerimiento. Complementa package.json para instalaciones reproducibles.

BACKEND/.env

Configuración privada local. server.js carga sus variables con dotenv. database.js consume DB_HOST, DB_PORT, DB_USER, DB_PASSWORD y DB_NAME; server.js consume PORT. No se reproducen valores secretos. Cambiar DB_NAME elige otra base, pero no la crea. Propuesta: entregar .env.example sin secretos reales y mantener .env fuera de Git.

node_modules y BACKEND/.npm-cache

Dependencias y caché generadas por herramientas. No son capas de la arquitectura de negocio. El node_modules de raíz observado es un enlace a un entorno de herramientas; el backend también tiene sus dependencias. No deben trasladarse como código propio ni editarse para reparar una funcionalidad.

Referencia 2. Arranque y configuración del backend

BACKEND/src/server.js

Importa dotenv antes de cargar app y database, para que la creación del pool pueda leer variables configuradas. Importa app.js y config/database.js. Convierte PORT a número o utiliza 3000.

handleServerError distingue puerto ocupado (EADDRINUSE) de otros errores de escucha, informa y termina el proceso. startServer espera database.checkConnection; imprime base y versión, luego ejecuta app.listen y registra el manejador error del servidor. Si MySQL falla, informa y termina sin abrir el puerto. La llamada final startServer() inicia todo.

Relación: package.json → server.js → app.js y database.js. Este archivo no contiene endpoints de ventas. Separarlo permite importar app para pruebas sin arrancar automáticamente un puerto.

BACKEND/src/app.js

Importa path, express, el agrupador routes y los manejadores de errores. Crea una instancia Express. frontendPath calcula una ruta absoluta desde __dirname hasta FRONTEND; no depende del lugar desde el que el usuario abre el navegador.

Orden de registro:

1. express.json interpreta cuerpos JSON.
2. express.static publica FRONTEND, sin seleccionar automáticamente index.html.
3. /api monta apiRoutes.
4. / entrega index.html y /app entrega app.html.
5. notFound atiende lo que no coincidió.
6. errorHandler recibe errores propagados.

Exporta app, no llama listen. La página /app puede descargarse sin token: la protección efectiva de datos está en las rutas API. FRONTEND/app.js además redirige si no encuentra sesión. Publicar HTML sin datos privados no equivale a publicar las tablas. BACKEND/database no se incluye en express.static.

BACKEND/src/config/database.js

Importa mysql2/promise. createPool prepara un conjunto reutilizable de conexiones: host, puerto, usuario, clave y base provienen del entorno, con valores alternativos. waitForConnections permite esperar una conexión; connectionLimit limita el pool a diez.

checkConnection ejecuta SELECT DATABASE() y VERSION(), y devuelve la primera fila. Se añade como propiedad del pool y se exporta ese objeto. Los cuatro modelos usan ese pool. Crear el pool no crea tablas; la primera consulta comprueba el acceso real. En ventas, getConnection reserva una conexión para que todas las sentencias pertenezcan a la misma transacción.

Referencia 3. Las seis rutas: el mapa entre URL y controlador

Una ruta indica verbo, dirección y manejadores. Cada archivo importa Express, el controlador correspondiente y, donde corresponde, `requireAuth`. Crea `express.Router` y lo exporta. No genera el HTML de la pantalla.

BACKEND/src/routes/index.js

Reúne `authRoutes` sin un prefijo adicional, `categoryRoutes` en `/categorias`, `productRoutes` en `/productos`, `saleRoutes` en `/ventas` y `administratorRoutes` en `/administradores`. `app.js` añade `/api` por fuera. Así, `router.get("/")` en `productRoutes` se convierte en `GET /api/productos`.

BACKEND/src/routes/authRoutes.js

`POST /login` llama `authController.login` sin sesión previa. `POST /logout` ejecuta primero `requireAuth` y después `logout`. Proteger `login` con una sesión obligatoria impediría obtener la primera sesión.

BACKEND/src/routes/categoryRoutes.js

`GET /` ejecuta `requireAuth` y `categoryController.list`. Es el único endpoint de categorías: no hay crear, editar ni eliminar categorías desde esta API.

BACKEND/src/routes/productRoutes.js

`router.use(requireAuth)` protege las rutas posteriores. `GET /` llama `list`; `POST /` llama `create`; `PUT /:id` llama `update`; `DELETE /:id` llama `remove`. El mismo sufijo `/:id` representa un producto, pero el verbo decide si editarlo o eliminarlo.

BACKEND/src/routes/saleRoutes.js

Protege todas sus operaciones. `GET /` llama `list`; `GET /resumen-categorias` llama `dailyCategorySummary`; `GET /:id` llama `detail`; `POST /` llama `create`. El resumen debe estar antes del parámetro genérico: de lo contrario la palabra `resumen-categorias` puede llegar a `detail` como si fuera un ID.

BACKEND/src/routes/administratorRoutes.js

Protege las operaciones mediante `router.use`. `GET /` lista, `POST /` crea y `PATCH /:id/estado` cambia activo. `PATCH` comunica una modificación parcial. No existen rutas para cambiar contraseña o editar nombre/correo de una cuenta.

Referencia 4. Middleware: comprobaciones compartidas

BACKEND/src/middleware/authMiddleware.js

Importa `sessionService` y exporta `requireAuth`. Lee `req.headers.authorization`. Si empieza por `Bearer` seguido de espacio, extrae el resto como token; de lo contrario usa `null`. Busca la sesión en el Map. Si falta, devuelve 401 y termina. Si existe, añade `req.token` y `req.user` y llama `next()`.

Relación: rutas → `requireAuth` → `sessionService.find`. Los controladores de venta y estado de administrador usan `req.user.id`. No consulta MySQL ni revisa si la cuenta fue desactivada después del login. Tampoco es JWT: el token es opaco, aleatorio y se busca en memoria.

BACKEND/src/middleware/errorMiddleware.js

`notFound` devuelve 404 con verbo y dirección de la petición no atendida. `errorHandler` tiene cuatro parámetros, firma que Express reconoce para errores. Convierte `ER_ROW_IS_REFERENCED_2` en 409; usa `error.status` si existe; de lo contrario 500. Para 500 escribe el error en consola y entrega un mensaje general. Para otros estados devuelve `error.message`.

Relación: controladores llaman `next(error)`; `app.js` registra este manejador al final. Una FK inválida al crear o un SIGNAL del trigger no están traducidos específicamente; pueden terminar en 500 aunque sean conflictos de datos.

Referencia 5. Servicios

BACKEND/src/services/sessionService.js

Importa `crypto` del propio Node. `sessions` es un Map compartido por los módulos que importan este archivo dentro del mismo proceso. `create` genera 24 bytes aleatorios, los convierte a 48 caracteres hexadecimales y asocia token con id, nombre y email. No guarda contraseña. Devuelve el token.

`find` devuelve la sesión o `null`; `remove` elimina la entrada. Exporta las tres funciones. `authController` crea/elimina, `authMiddleware` consulta. No hay fecha de vencimiento, persistencia ni función para eliminar todas las sesiones de una cuenta.

BACKEND/src/services/measurementService.js

`KG_PER_LB = 0.45359237` es la equivalencia utilizada por el código. `convertToBaseQuantity` recibe cantidad solicitada, unidad de venta y unidad base del producto.

Si la base es unidad, solo acepta unidad y cantidad entera; devuelve `NaN` si es incompatible. Para peso acepta g, kg y lb. Lleva primero a kilogramos: `g/1000`, `lb×factor` o kg sin cambio. Si la base es lb, divide los kilogramos por el factor. Redondea el resultado a tres decimales y lo devuelve como `Number`.

`saleModel.create` llama este servicio y comprueba además positividad y existencia. `NaN` es una señal numérica de conversión inválida, no una respuesta HTTP. La función no realiza `SELECT` ni `UPDATE`. El frontend reproduce una conversión parecida, pero no aplica siempre ese mismo redondeo.

Referencia 6. Controladores: contenido completo por función

BACKEND/src/controllers/authController.js

Importa administratorModel y sessionService. login obtiene email, elimina espacios y lo pasa a minúsculas; toma password. Si faltan, responde 400. Espera findActiveByCredentials; si no encuentra cuenta, responde 401. Si encuentra, responde un objeto token creado por sessionService. catch entrega errores a next.

logout elimina req.token del Map y responde confirmación. requireAuth ya debió llenar req.token. Exporta login y logout. No pide confirmación de contraseña en la API: esa repetición pertenece solo a login.js.

BACKEND/src/controllers/categoryController.js

Importa categoryModel. Su única función list espera findAll y envía las filas JSON. Si falla, next(error). Es pequeño porque esta consulta no recibe filtros ni escribe datos.

BACKEND/src/controllers/productController.js

Importa productModel. Tiene tres auxiliares y cuatro operaciones:

parseProduct(body)

Qué hace: Limpia nombre/corte; convierte categoría, precio y stock a números; copia unidad; usa activo recibido o true cuando es null/undefined. Corte vacío pasa a null.

validateProduct(product)

Qué hace: Exige nombre, categoría entera positiva, unidad permitida, precio finito >0 y stock finito >=0. Devuelve booleano.

parseId(value)

Qué hace: Devuelve entero positivo o null.

list

Qué hace: Construye search, categoryId y lowStock desde query; llama findAll y devuelve filas. stockBajo debe ser el texto 'true'.

create

Qué hace: Interpreta y valida body; si falla 400; si pasa INSERT mediante modelo; devuelve 201, id y mensaje.

update

Qué hace: Valida ID y datos; llama update; si no afecta filas devuelve 404; si afecta, confirma.

remove

Qué hace: Valida ID; llama remove; responde 404 si no afecta filas y confirmación en éxito.

Exporta solo list/create/update/remove; los auxiliares quedan internos. Todas las operaciones asíncronas propagan errores con next. La validación no comprueba todavía tipo string antes de trim, longitudes máximas ni que activo sea booleano. La categoría puede ser numéricamente válida e inexistente: la FK de MySQL detecta ese caso.

BACKEND/src/controllers/saleController.js

Importa saleModel. parseDate acepta ausencia como null y comprueba formato YYYY-MM-DD; no valida que el día exista en el calendario. parseCategoryId acepta ausencia como null y exige entero positivo en otro caso.

list interpreta req.query.fecha y categoria, responde 400 si alguno es false y llama findAll({date,categoryId}). dailyCategorySummary valida fecha y llama findDailyCategorySummary. detail valida req.params.id, llama findByid y responde 404 si no hay venta.

create extrae items, cliente_nombre, metodo_pago y monto_recibido. El método por defecto es efectivo. validItems exige arreglo no vacío y valida cada ID, cantidad positiva finita y unidad opcional permitida. Luego comprueba efectivo/tarjeta/transferecia. Llama saleModel.create con los datos normalizados y administradorId tomado de req.user.id; responde 201 con resultado y mensaje.

El controlador no acepta precio ni total del navegador como autoridad. El modelo vuelve a consultar precio y stock. items.every con propiedades mal tipadas aún puede generar excepciones; falta validación estructural completa.

BACKEND/src/controllers/administratorController.js

Importa administratorModel. list devuelve findAll. create limpia nombre y email, normaliza correo y exige clave con longitud mínima de seis; llama create y responde 201. ER_DUP_ENTRY se traduce a 409 por correo repetido; otros errores se propagan.

updateStatus convierte ID y exige req.body.activo booleano. Si pretende desactivar el mismo ID de req.user, responde 400. Llama setStatus y devuelve 404 si no afecta filas o mensaje de activación/desactivación si funciona.

Esta protección evita autodesactivarse, pero no es un sistema de roles. Todas las sesiones válidas pueden acceder al módulo. Desactivar otra cuenta no llama sessionService.remove.

Referencia 7. Modelos: qué significa cada consulta

BACKEND/src/models/categoryModel.js

Importa pool. findAll ejecuta SELECT id,nombre FROM categorias ORDER BY nombre; devuelve filas. Usa query sin parámetros porque no incorpora entradas del usuario. ORDER BY explica por qué el primer filtro no necesariamente corresponde al ID 1. categoryController lo consume.

BACKEND/src/models/administratorModel.js

findActiveByCredentials ejecuta SELECT id,nombre,email con email parametrizado, password_hash = SHA2 de la clave y activo = TRUE. Devuelve primera coincidencia o null. La API no recibe el hash.

findAll lista id, nombre, email, activo y fecha_creacion. ORDER BY activo DESC,nombre sitúa activos primero. create inserta nombre, email, hash SHA2 y activo verdadero; devuelve insertId. setStatus actualiza activo por ID; devuelve affectedRows.

Los ? mantienen separados valores y consulta. El hash actual no utiliza un algoritmo adaptativo para contraseñas: es una deuda conocida. El archivo no envía correos ni genera claves temporales aleatorias.

BACKEND/src/models/productModel.js

findAll recibe `search`, `categoryId` y `lowStock`. Forma `%texto%` para `LIKE`. `SELECT` obtiene campos del producto y el nombre de su categoría mediante `INNER JOIN`. La búsqueda considera nombre o corte; una condición permite omitir categoría si es `null`; otra permite omitir stock bajo si es `false`. Cuando se activa exige `p.stock <=10`. Ordena por nombre.

Devuelve activos e inactivos: `visibleProducts` del frontend de ventas hace el filtro comercial. La búsqueda del inventario ocurre en MySQL; la búsqueda del catálogo cargado ocurre en navegador.

`create` inserta siete campos: `nombre`, `tipo_corte`, `categoria_id`, `unidad_medida`, `precio`, `stock` y `activo`. Devuelve `insertId`. `update` reemplaza esos siete por ID y devuelve `affectedRows`. `remove` ejecuta `DELETE` por ID y devuelve `affectedRows`. Borrar no es poner `activo=false`. Si un producto ya está en `detalle_venta`, `ON DELETE RESTRICT` lo protege.

BACKEND/src/models/saleModel.js

Importa `pool` y `convertToBaseQuantity`. Combina consultas de lectura y coordinación de la venta.

findAll: construye una condición opcional por categoría; solo añade su parámetro cuando la condición está presente. Une ventas con administradores, detalles y productos mediante `LEFT JOIN`. Selecciona datos de cabecera, nombre de quien atendió, `COUNT(d.id)` como `items` y `SUM(d.subtotal)` como `total_filtrado`. `COALESCE` cambia suma `null` a cero. Filtra fecha si existe y estado completada. `GROUP BY` reúne filas de una misma venta; `ORDER BY` fecha `DESC` muestra recientes primero. Con categoría, las líneas de otras categorías quedan fuera de la suma y del conteo.

findDailyCategorySummary: comienza en categorías y hace `LEFT JOIN` hacia productos, detalles y ventas, de manera que las categorías sin ventas aparezcan con cero. `CASE` suma `subtotal/cantidad` solo si la venta es completada y del día solicitado. `COALESCE(fecha,CURDATE())` usa hoy de MySQL si falta fecha. Devuelve `total_vendido` y `unidades_vendidas`. La segunda suma mezcla unidades base de productos distintos, una limitación del reporte actual.

findById: primera consulta recupera cabecera y `atendido_por`. Si no existe retorna `null`. Segunda consulta recupera detalles con nombres, categoría y unidad del producto actual. Devuelve `{...cabecera,items}`. Guarda el precio histórico desde `d.precio_unitario`, pero `nombre/categoría/unidad` provienen del catálogo actual. Esta diferencia explica el problema histórico al editar productos.

create: recibe `items`, `clienteNombre`, `metodoPago`, `montoRecibido` y `administradorId`. Su secuencia es:

1. Reserva `connection` y comienza transacción.
2. Para cada artículo consulta producto activo con `FOR UPDATE`. Ese bloqueo evita que otra transacción modifique simultáneamente la fila bloqueada hasta terminar.
3. Convierte cantidad a unidad base; si falta unidad de venta usa la del producto. Rechaza producto inexistente/inactivo, conversión inválida, cantidad `<=0` o stock insuficiente con `error.status=400`.
4. Suma `precio de MySQL × cantidad base`, redondea total a dos decimales y acumula `validatedItems`.
5. En efectivo exige recibido finito y suficiente; en tarjeta/transferencia usa total como recibido. Calcula cambio a dos decimales.
6. Inserta ventas con estado completada y cliente o Consumidor final.
7. Inserta cada detalle con cantidad y precio. Los triggers actúan aquí; el subtotal es una columna generada.

8. Inserta movimientos con tipo pago por el total.
9. commit confirma y devuelve id,total,monto_recibido,cambio.
10. catch ejecuta rollback y relanza el error; finally devuelve la conexión al pool mediante release.

La reserva de conexión ocurre antes del try: si no logra reservarla, el error sube al controlador sin una conexión que liberar. Las garantías transaccionales requieren tablas con motor transaccional, como InnoDB. El archivo no implementa cancelaciones ni devoluciones.

Referencia 8. FRONTEND: los dos documentos HTML

FRONTEND/index.html

doctype selecciona HTML moderno; lang=es indica idioma. meta charset permite caracteres como ñ; viewport adapta el ancho a dispositivos. title nombra la pestaña y link carga /css/styles.css.

main contiene login-form. Sus campos son email, password y password-confirmation. type=email y required activan validaciones nativas. Los atributos autocomplete y los data-* dirigidos a gestores expresan preferencias de autocompletado; el comportamiento final depende del navegador/gestor.

Los botones con data-password-target apuntan al ID del campo correspondiente; type=button evita enviar el formulario. Los SVG dibujan los iconos del ojo. login-error muestra errores y aria-live anuncia cambios a tecnologías de asistencia. El botón submit dispara el formulario. script type=module carga /js/login.js, que importa api.js.

Si un ID cambia solo en HTML y el JS conserva el anterior, querySelector devuelve null y puede fallar la inicialización. Las credenciales no están escritas en el HTML.

FRONTEND/app.html

Carga el mismo CSS y /js/app.js como módulo. Tiene cabecera, menú, espacio de mensajes, cinco secciones y cuatro diálogos.

logout

Contenido: Cerrar sesión **Archivo que lo maneja:** app.js → api.js

data-section en menú/tarjetas

Contenido: Identifica home, inventory, sales, history, administrators **Archivo que lo maneja:** app.js
openSection

message

Contenido: Mensajes temporales **Archivo que lo maneja:** ui.js showMessage

home-section

Contenido: Bienvenida y cuatro indicadores **Archivo que lo maneja:** dashboard.js

stat-products/categories/low-stock/sales

Contenido: Valores numéricos; las tarjetas tienen navegación **Archivo que lo maneja:** dashboard.js y app.js

inventory-section

Contenido: Pestañas, resumen diario, filtros y tabla **Archivo que lo maneja:** inventory.js

inventory-category-tabs

Contenido: Botones generados por categorías **Archivo que lo maneja:** renderCategoryTabs

inventory-summary-*

Contenido: Categoría, fecha, total y cantidad **Archivo que lo maneja:** renderCategorySummary

search, category-filter, low-stock

Contenido: Filtros de consulta **Archivo que lo maneja:** loadInventory

product-rows

Contenido: Cuerpo vacío que se llena con filas y botones **Archivo que lo maneja:** renderProducts

sales-section

Contenido: Catálogo y carrito **Archivo que lo maneja:** sales.js

sale-search, sale-category-filter

Contenido: Filtros locales del catálogo **Archivo que lo maneja:** visibleProducts y renderCatalog

catalog, sale-result-count, sale-page-label

Contenido: Tarjetas y paginación **Archivo que lo maneja:** renderCatalog

sale-previous-page, sale-next-page

Contenido: Cambian página **Archivo que lo maneja:** initializeSales

sale-customer, payment-method, amount-received

Contenido: Datos del cobro **Archivo que lo maneja:** updatePayment y save-sale

cart-items, cart-total, sale-change, save-sale

Contenido: Carrito y confirmación **Archivo que lo maneja:** renderCart y eventos

history-section

Contenido: Filtros, resumen e historial **Archivo que lo maneja:** loadSalesHistory

history-date-filter, history-category-filter

Contenido: Parámetros de la consulta **Archivo que lo maneja:** loadSalesHistory

history-category-total/units, history-sale-count/total-column

Contenido: Indicadores y encabezado según filtro **Archivo que lo maneja:** loadSalesHistory

sale-rows

Contenido: Ventas y botones de recibo **Archivo que lo maneja:** loadSalesHistory

administrators-section, administrator-rows

Contenido: Lista y acciones de cuentas **Archivo que lo maneja:** administrators.js

product-dialog/product-form

Contenido: Crear/editar producto en un mismo formulario **Archivo que lo maneja:** inventory.js

administrator-dialog/administrator-form

Contenido: Crear una cuenta **Archivo que lo maneja:** administrators.js

receipt-dialog/receipt-content

Contenido: Comprobante y controles de impresión/cierre **Archivo que lo maneja:** showReceipt

daily-report-dialog/daily-report-content

Contenido: Resumen diario imprimible **Archivo que lo maneja:** openDailyReport

product-id es hidden y distingue editar de crear. product-name/cut/category/unit/price/stock/active corresponden a las propiedades del payload. Los límites HTML ayudan al usuario; el backend necesita validación propia. La existencia acepta incrementos de 0.001 incluso para unidad: esa restricción aún no es coherente en todo el sistema.

Los elementos dialog usan `showModal` y `close` desde JavaScript. `receipt-content` se completa dinámicamente e incluye un selector que no existe hasta consultar un recibo. `hidden` controla visibilidad de secciones; no es un permiso de acceso.

Referencia 9. FRONTEND/js/api.js: comunicación común

`TOKEN_KEY` define la clave `districarnes_session`. `hasSession` devuelve si existe un valor en `sessionStorage`, sin consultar al servidor.

`apiRequest(path,{method='GET',body}={})` admite omitir opciones. Lee token, prepara `Accept: application/json` y `Authorization` si existe. Añade `Content-Type` solo cuando hay body. `fetch` llama `/api${path}` y `JSON.stringify` convierte el objeto a texto JSON. Intenta interpretar respuesta JSON; si no puede, usa un objeto vacío.

Si recibe 401 y había token, lo elimina y redirige a `/`. Si `response.ok` es falso lanza `Error` con el mensaje disponible. Si es exitoso devuelve los datos. Un fallo de red también rechaza la promesa y debe manejarlo quien llama.

`startSession` llama `POST /login` con `email/password` y guarda `session.token`. `closeSession` intenta `POST /logout` y, en `finally`, elimina la copia local y redirige. Importadores: `login.js` y `app.js` para sesión; todos los módulos de datos usan `apiRequest`. Centralizarlo evita repetir `fetch` y cabeceras.

Referencia 10. FRONTEND/js/ui.js: utilidades visuales

`$` abrevia `document.querySelector`. `formatMoney` usa `Intl.NumberFormat` con `es-CO` y `COP`, sin decimales visibles; formatea, no calcula impuestos ni redondea el dato almacenado. `formatDateTime` usa fecha/hora local. `formatDate` agrega `T00:00:00` a cadenas `YYYY-MM-DD` antes de convertirlas, evitando interpretarlas directamente como medianoche UTC.

`today` calcula la fecha del navegador con su desfase horario y devuelve `YYYY-MM-DD`. `formatQuantity` muestra hasta tres decimales y cambia unidad por `und`. `escapeHtml` reemplaza `&`, `<`, `>`, comillas dobles y simples por entidades; sirve para interpolar texto dentro de HTML. No sustituye validación de URL ni parametrización SQL.

`messageTimer` conserva el temporizador previo. `showMessage` lo cancela, escribe texto con `textContent`, asigna clase `success/error` y borra el aviso tras 3500 ms. Depende de `#message`, presente en `app.html`. `login.js` tiene su propio elemento de error.

Referencia 11. FRONTEND/js/login.js

Importa `hasSession` y `startSession`. Si hay token local redirige a `/app`. Obtiene referencias al formulario, error, botón, tres campos y botones de ojo.

`clearLoginFields` reinicia el formulario, vacía valores, oculta ambas claves, restablece clases/títulos/etiquetas accesibles y borra errores. Se llama inmediatamente, en `pageshow` y mediante `requestAnimationFrame`. Son intentos de mantener los campos vacíos ante restauración de la página.

Cada botón de ojo encuentra el campo por `dataset.passwordTarget`, alterna `type text/password` y actualiza `active`, `aria-label` y `title`.

El listener `submit` evita recarga, borra error, compara las dos claves y detiene si no coinciden. Deshabilita el botón, espera `startSession` y redirige al panel. Si falla muestra mensaje y habilita el botón. La segunda clave no se envía al backend. No hay registro de cuentas en esta página.

Referencia 12. FRONTEND/js/app.js: coordinación del panel

Importa API/sesión, inicializadores y cargadores de módulos, y utilidades. `sectionLoaders` empieza vacío.

`openSection(name)` marca `active` en el menú, muestra la sección coincidente y oculta las demás. Ejecuta la función registrada en `sectionLoaders[name]`, desplaza al inicio y muestra errores si falla. No descarga otra página para cada sección.

`initializeApplication` consulta categorías una vez, inicializa inventario/ventas/administradores y asigna funciones a `home/inventory/sales/history/administrators`. Registra un listener en `document` que busca el ancestro más cercano con `data-section`; puede aplicar categoría o stock bajo antes de abrir la sección. Conecta `logout` y abre `home`.

Al final, `hasSession` decide inicializar o volver al acceso. `initialize` prepara eventos; `load` recupera datos. Repetir `initialize` sin control duplicaría listeners. Las pestañas de categoría del inventario tienen su propio listener; no dependen de `data-section`.

Referencia 13. FRONTEND/js/dashboard.js

Importa `apiRequest`, `$`, `today`. `loadDashboard(categories)` calcula fecha local y consulta productos y ventas del día con `Promise.all`. Cuando ambas llegan, escribe longitud de productos, longitud de categorías, cantidad de productos `stock <= 10` y longitud de ventas.

`Promise.all` ejecuta ambas solicitudes sin esperar que termine la primera. Si una falla, la operación conjunta rechaza. No hay endpoint `/dashboard`; estos indicadores se calculan a partir de endpoints existentes. Incluye productos inactivos porque `/productos` los devuelve.

Referencia 14. FRONTEND/js/inventory.js

Importa API y utilidades. `products` conserva la última lista filtrada, `inventoryCategories` las categorías iniciales.

renderCategoryTabs

Explicación y relación: Lee `category-filter`, genera botones con `data-inventory-category` y marca `active`. Escapa nombre.

renderCategorySummary(summary)

Explicación y relación: Encuentra categoría seleccionada y resumen correspondiente; rellena etiquetas, fecha, total y unidades base. Usa cero si no hay datos.

selectInventoryCategory(id,load=true)

Explicación y relación: Comprueba que exista, cambia selector, redibuja pestañas y opcionalmente

consulta inventario. La usa también `app.js`.

`selectLowStock(enabled,load=true)`

Explicación y relación: Cambia checkbox y consulta opcionalmente; sirve a accesos del tablero.

`productPayload`

Explicación y relación: Lee los siete campos editables y prepara objeto para API; checkbox aporta booleano.

`renderProducts`

Explicación y relación: Genera filas con datos, unidades, estado y botones data-edit/data-delete. Marca stock bajo; muestra mensaje si lista vacía.

`loadInventory({refreshSummary=true}={})`

Explicación y relación: Forma query con `URLSearchParams`. Consulta productos y opcionalmente resumen diario en paralelo; conserva `products` y redibuja.

`openProductForm(product)`

Explicación y relación: Reinicia formulario; si recibe producto rellena y coloca ID, si no deja valores de creación; abre diálogo.

`initializeInventory(categories)`

Explicación y relación: Conserva categorías, llena selects, selecciona primera, dibuja pestañas y registra todos los eventos.

Los eventos de `initializeInventory`: Nuevo abre formulario; Cerrar cierra; escribir búsqueda recarga sin resumen; cambiar categoría actualiza pestañas y carga; pulsar pestaña selecciona categoría; stock bajo recarga sin resumen.

Submit lee `product-id`: sin ID hace `POST /productos`; con ID hace `PUT /productos/ID`. Tras éxito cierra, avisa y recarga productos. El listener de `product-rows` usa `data-edit` para encontrar el objeto en `products` y `data-delete` para pedir confirmación y enviar `DELETE`. Los botones se generan dinámicamente, por eso se escucha al contenedor.

No todas las recargas de filtros tienen `catch local`; una respuesta de búsqueda antigua podría llegar después de otra reciente. Son detalles de manejo asíncrono pendientes, no cambios en la relación modelo-controlador.

Referencia 15. `FRONTEND/js/administrators.js`

Importa `apiRequest` y utilidades. `renderAdministrators` genera nombre/correo/estado/fecha y un botón por cuenta con `data-admin-id` y `data-next-active`. Ese segundo atributo es texto `'true'` o `'false'`. No conserva la lista en una variable global del módulo.

`loadAdministrators` consulta `GET /administradores` y dibuja. `initializeAdministrators` conecta apertura/cierre del diálogo, submit y listener de tabla.

Submit envía nombre,email,password a `POST /administradores`; si funciona limpia formulario, cierra, avisa y recarga. El listener de tabla convierte ID a número y `nextActive` a booleano con comparación de texto; envía `PATCH /administradores/ID/estado` y recarga. La restricción de autodesactivación se aplica en backend, aunque el botón se muestre.

Referencia 16. FRONTEND/js/sales.js: cada función y cada evento

Es el módulo con más responsabilidades. Importa `apiRequest` y utilidades. `PAGE_SIZE=24` fija tarjetas por página; `KG_PER_LB` fija conversión. `catalogProducts` es catálogo descargado, `cart` es compra aún no guardada, `currentPage` indica página y `dailyReport` guarda datos preparados para imprimir.

Catálogo

`categoryImage` transforma categoría a minúsculas, normaliza acentos y construye `/images/nombre.png`. Así `Vísceras` usa `visceras.png`. Si se crea otra categoría, debe existir su imagen o definirse `imagen_url` en el producto.

`visibleProducts` lee búsqueda/categoría de la venta y filtra `catalogProducts`: activo, `stock>0` y coincidencias. Busca en nombre, corte, categoría. `renderCatalog` calcula total de páginas con `Math.ceil`, ajusta `currentPage`, obtiene slice de 24, actualiza contadores/botones y crea tarjetas con imagen, precio, stock y `data-add-product`. Usa `imagen_url` si existe; de otro modo `categoryImage`.

`loadSaleCatalog` consulta `/productos`, reinicia página a 1 y redibuja. No limpia `cart` ni actualiza automáticamente las copias de stock/precio de artículos que ya estaban allí; el servidor volverá a validar al guardar.

Medidas y estado del carrito

`quantityInBase(item)` transforma cantidad a unidad base. En unidad exige entero; para peso convierte por gramos/kilogramos/libras. No aplica el redondeo de tres decimales del servicio backend.

`cartTotal` reduce las líneas a suma `precio×cantidad` base; aporta cero si una conversión no es finita. `cartIsValid` exige al menos una línea y que todas tengan cantidad válida, positiva y no superior al stock de la copia local.

`convertDisplayedQuantity` cambia la cantidad escrita al seleccionar otra unidad, intentando conservar el peso: primero pasa a kg y luego a destino; gramos se redondean a entero y kg/lb a tres decimales. `saleUnitOptions` genera opciones: solo Unidad para productos por unidad, o g/kg/lb para peso.

`renderCart` crea las líneas con precio, stock, input de cantidad, selector de unidad y botón Quitar. Los `data-*` identifican producto. `min/step` orientan la entrada. Muestra aviso cuando cantidad supera stock, dibuja total, habilita/deshabilita `save-sale` según `cartIsValid` y llama `updatePayment`.

Cobro

`updatePayment` detecta si es efectivo, muestra/oculta campos, toma recibido escrito o total para otros métodos y dibuja cambio con mínimo visual cero. Mostrar cero no demuestra que el recibido alcance: el servidor lo comprueba. El botón se habilita según carrito, no según suficiencia del efectivo.

Historial

`loadSalesHistory` toma fecha del filtro o today, toma categoría y solicita ventas filtradas y resumen diario en paralelo. Si hay categoría elige su resumen; si no, reduce resúmenes de todas. Crea `dailyReport` con fecha, nombre de categoría, ventas, total y unidades.

Actualiza indicadores, cambia encabezado entre Total y Subtotal categoría y dibuja sale-rows. Usa `sale.total_filtrado`, por lo que una venta de varias categorías muestra solo su parte seleccionada. La cantidad de productos mostrada procede del conteo de líneas SQL.

`openDailyReport` usa `dailyReport` ya cargado: no vuelve a consultar ni escribir en MySQL. Compone encabezado, filtros, tabla y total y abre diálogo. Si los datos cambiaron después de cargar historial, el reporte conserva aquella consulta hasta recargar.

Recibo

`showReceipt(id)` consulta `/ventas/ID`. Usa Map para construir categorías sin repetir. Genera cabecera con cliente, empleado, fecha y pago; inserta selector, tabla, subtotal visible, total, recibido y cambio.

Dentro define `renderReceiptItems`, que recuerda sale mediante el alcance de la función. Filtra sus items por categoría, dibuja filas y suma subtotal visible. Registra el cambio del selector, dibuja la primera vez y abre receipt-dialog. Filtrar no cambia total guardado ni crea otra venta.

Todos los eventos de `initializeSales(categories)`

Llena selects de catálogo e historial y coloca la fecha actual. Después registra:

input de sale-search / change de sale-category-filter

Cambio que produce: Reinicia página y filtra catálogo local.

Click anterior/siguiente

Cambio que produce: Cambia `currentPage` y redibuja.

Click en catalog

Cambio que produce: Lee `data-add-product`, busca producto; si no está en cart lo copia con 1 unidad o 400 g. Si ya está, no incrementa. Redibuja carrito.

input de cart-items

Cambio que produce: Lee `data-cart-quantity`, actualiza cantidad, total, botón y cambio. No redibuja toda la línea en ese evento.

change de cart-items

Cambio que produce: Lee `data-cart-unit`, convierte cantidad mostrada, cambia unidad y redibuja carrito.

Click Quitar

Cambio que produce: Filtra cart excluyendo ID y redibuja; todavía no hay stock que devolver.

Cambio de método / input de recibido

Cambio que produce: Recalcula presentación del pago.

Cambio de fecha/categoría del historial

Cambio que produce: Consulta nuevamente historial.

Abrir reporte / cerrar reporte / imprimir reporte

Cambio que produce: Usa datos preparados, cierra diálogo o llama `window.print`.

Click save-sale

Cambio que produce: Envía operación de venta al servidor.

Click en sale-rows

Cambio que produce: Lee `data-receipt` y llama `showReceipt`.

Cerrar/imprimir recibo

Cambio que produce: Cierra diálogo o llama `window.print`.

El manejador save-sale crea items con producto_id,cantidad,unidad_venta. Agrega cliente, método y recibido; envía POST /ventas. En éxito vacía cart, cliente y recibido; redibuja; informa ID/total devueltos; recarga catálogo; consulta recibo. En error muestra mensaje. No deshabilita expresamente el botón mientras espera, por lo que existe riesgo de doble envío. Al final de initializeSales, renderCart prepara la pantalla vacía.

Propuesta: separar este archivo en catálogo, carrito/medidas, historial y comprobantes, con un coordinador. Antes de mover funciones, definir quién conserva cart y cómo se comparten datos, para evitar múltiples carritos independientes.

Referencia 17. FRONTEND/css/styles.css: bloques y relación con HTML/JS

CSS define apariencia y disposición; no consulta MySQL. Un selector .clase busca class; #id busca id; :hover es un estado; > exige hijo directo; [open] comprueba atributo. Las reglas posteriores pueden sobrescribir anteriores según especificidad y orden.

:root

Qué controla: Fuente, colores generales y variables --primary/--dark/--border. var(...) reutiliza esas variables.

*, body, button/input/select, h1/h2/p

Qué controla: Modelo de caja border-box, margen inicial, tipografía y base de controles/textos.

button y button:disabled

Qué controla: Apariencia y opacidad; la desactivación real viene del atributo disabled, no del color.

.hidden

Qué controla: display:none!important; app.js lo alterna para navegación.

.login-page/.login-card/.password-field/.password-eye

Qué controla: Centrado de acceso, tarjeta, espacio para ojo y estados del botón. Las reglas svg dibujan trazo y tamaño.

.logo/.logo.small/.overline/.muted

Qué controla: Marca, variantes pequeñas y jerarquía visual.

label/input/select/.check/.two-cols

Qué controla: Formularios, casillas y distribución de campos.

.card/.title-row/.filters/.table-wrap/th/td

Qué controla: Tarjetas, títulos, filtros, desplazamiento horizontal y tablas.

.stock/.stock.low

Qué controla: Etiquetas normales y de alerta que renderProducts y administradores asignan.

.actions/.link/.danger/.empty

Qué controla: Acciones compactas, botón de enlace, eliminar y estados sin datos.

.message/.error/.success/.message.error

Qué controla: Espacio y colores de avisos; showMessage cambia clases y texto.

dialog/dialog::backdrop/.dialog-form/.dialog-head/.icon

Qué controla: Ventanas, fondo oscurecido, cabecera y cierre.

.sales-layout/.catalog/.product-card y descendientes

Qué controla: Distribución de venta y tarjetas con imágenes recortadas mediante object-fit.

.sale-cart/.cart-item/.cart-total/.weight-controls/.cart-warning

Qué controla: Carrito, controles y avisos; sticky mantiene carrito visible en escritorio.

.app-page/.topbar/.brand/.app-shell/.sidebar/.side-nav/.workspace

Qué controla: Marco principal, cabecera fija al desplazarse, menú y espacio de contenido.

.menu-label/.sidebar-help/.nav-button.active

Qué controla: Texto auxiliar y estado de navegación que app.js modifica.

.welcome-card/.welcome-actions/.welcome-mark

Qué controla: Bienvenida, botones y letra decorativa.

.stats-grid/.stat-card y estados

Qué controla: Cuadrícula e interacción de indicadores.

.category-tabs/.category-tab/.inventory-category-row/.category-daily-card

Qué controla: Pestañas y resumen por categoría; active indica seleccionada.

.sale-filters/.result-count/.history-filters/.history-total/.change-row

Qué controla: Filtros, conteos, total de historial y cambio de efectivo.

.receipt-dialog/.receipt/.receipt-head/dl/.receipt-totals/.receipt-filter/.receipt-filter-total/.receipt-thanks

Qué controla: Diseño del recibo y subtotal filtrado.

.daily-report-dialog/.daily-report/.daily-report-head/.daily-report-meta/.daily-report-total

Qué controla: Resumen diario imprimible.

.receipt-actions/.outline-dark/.outline-button/.secondary

Qué controla: Variantes de botones y barra de acciones.

.catalog-pagination y descendientes

Qué controla: Alineación, espaciado y tamaños de navegación de páginas.

Las unidades rem dependen del tamaño de fuente raíz, px fija píxeles CSS, vw/vh representan proporción de ventana. Grid distribuye filas/columnas; flex organiza elementos sobre un eje. min/max/clamp limitan tamaños sin necesitar una regla por cada pantalla.

@media hasta 950 px reduce columnas de indicadores y oculta marca decorativa. Hasta 800 px transforma menú a horizontal, elimina la columna lateral de layout, apila filtros/venta, ajusta títulos/formularios y deja carrito sin sticky. Hasta 560 px ajusta tipografías, botones y paginación.

@media print oculta mediante visibility el contenido general, vuelve visibles los diálogos de recibo/reporte abiertos, les da posición/tamaño de impresión y elimina .receipt-actions. window.print activa el flujo del navegador; CSS selecciona qué se ve. No se genera un PDF mediante backend: el usuario puede elegir guardar PDF en el diálogo del navegador.

Referencia 18. FRONTEND/images: los cinco archivos

res.png, cerdo.png, pollo.png, pescado.png y visceras.png son imágenes de categoría, no código. sales.js construye sus URL con categoryImage. app.js del backend las entrega mediante express.static. La base tiene imagen_url opcional por producto, que prevalece sobre la imagen de categoría; el formulario actual de inventario no edita ese campo.

No es necesario memorizar bytes PNG para defender su función. Si falta una imagen, primero comparar nombre de categoría normalizado, nombre de archivo y URL solicitada en Network.

Referencia 19. BACKEND/database/schema.sql: cada bloque, tabla y regla

Inicio y reinicio

CREATE DATABASE IF NOT EXISTS crea la base si falta; utf8mb4 permite caracteres Unicode y unicode_ci define comparación sin distinción de mayúsculas dentro de esa intercalación. USE selecciona la base de las siguientes sentencias.

SET FOREIGN_KEY_CHECKS=0 desactiva comprobación de relaciones durante el borrado. Siete DROP TABLE eliminan tablas previas; después se reactiva. IF EXISTS solo evita error cuando la tabla falta: no conserva datos. Por tanto, ejecutar todo el archivo es reconstrucción, no una migración conservadora.

Vocabulario SQL del archivo

INT UNSIGNED representa entero sin signo; BIGINT permite IDs mayores; AUTO_INCREMENT asigna ID automáticamente; PRIMARY KEY identifica de forma única. VARCHAR(n) limita longitud; NOT NULL exige valor; UNIQUE impide duplicados. DECIMAL(10,2) reserva diez dígitos totales y dos decimales; DECIMAL(10,3), tres. BOOLEAN representa verdadero/falso; ENUM restringe alternativas. DEFAULT aporta valor omitido. TIMESTAMP registra fecha/hora, y ON UPDATE CURRENT_TIMESTAMP actualiza la marca al cambiar la fila.

FOREIGN KEY enlaza una columna con otra tabla. RESTRICT impide borrar una fila referenciada; CASCADE propaga la operación; SET NULL conserva la fila dependiente quitando la referencia. Los nombres fk_* identifican restricciones, no funciones JavaScript.

administradores

id identifica la cuenta; nombre es descripción; email es obligatorio y único; password_hash conserva hash, no clave recuperable; activo permite bloquear nuevos ingresos; fecha_creacion se completa automáticamente. administratorModel lee/escribe esta tabla y ventas la referencia con administrador_id.

categorias

id, nombre único y fecha_creacion. categoryModel devuelve solo id/nombre. productos.categoria_id la referencia con restricción de borrado. No existe campo imagen en categoría; la imagen se deduce en frontend.

productos

id; nombre; tipo_corte opcional; categoria_id obligatorio; imagen_url opcional; unidad_medida kg/lb/unidad con kg por defecto; precio; stock con tres decimales; activo; fecha_registro y fecha_actualizacion automática. Precio y stock están expresados en unidad_medida. La FK actualiza referencias en cascada y restringe borrar categoría usada.

ventas

id; administrador_id opcional; cliente_nombre con Consumidor final por defecto; metodo_pago efectivo/tarjeta/transfencia; monto_recibido; cambio; fecha automática; total; estado pendiente/completada/cancelada. Al borrar administrador, SET NULL conserva su venta. El código crea directamente completada; no implementa una operación para pasar por todos los estados.

detalle_venta

id identifica la línea; venta_id su cabecera; producto_id el artículo; cantidad en unidad base; precio_unitario histórico; subtotal GENERATED ALWAYS AS cantidad*precio_unitario STORED. La columna se calcula en MySQL, por lo que el INSERT del modelo no envía subtotal. Borrar cabecera propaga borrado de detalles; borrar producto vendido se restringe. No hay trigger que reponga automáticamente stock por eliminar una venta: no confundir CASCADE con una devolución.

movimientos

id; tipo pago/devolucion/cancelacion/actualizacion; monto; fecha; venta_id obligatorio. Su FK elimina movimientos si se borra venta. El modelo únicamente inserta tipo pago. Esta tabla no registra entradas/salidas de unidades de inventario.

auditoria_productos

id BIGINT; producto_id; accion ACTUALIZACION/ELIMINACION; nombre_anterior; precio_anterior/precio_nuevo; stock_anterior/stock_nuevo; usuario_bd; fecha. No tiene FK al producto, permitiendo conservar el identificador de uno borrado. No registra creación ni almacena directamente ID del administrador web.

Datos iniciales

INSERT de categorías crea Res, Cerdo, Pollo, Pescado y Vísceras. INSERT de administradores crea cuentas de demostración con hash SHA2; sus claves de ejemplo aparecen en el SQL y no se repiten en este manual. No son configuración segura para un despliegue real.

INSERT de productos define nombre,corte,categoría,precio,stock; como omite unidad_medida, usa kg incluso cuando el nombre contiene una presentación comercial. El bloque ampliado utiliza UNION ALL para formar un catálogo y CROSS JOIN con cuatro presentaciones. CONCAT forma nombres, se suma incremento de precio y se toma stock de la presentación. WHERE NOT EXISTS comprueba nombre/categoría antes de insertar esa combinación; esto no vuelve seguro el reinicio anterior por DROP TABLE.

Un nombre con «500 g» no convierte automáticamente el artículo a paquete ni cambia su unidad base. Esa distinción debe explicarse si el profesor pregunta por esos datos.

Triggers y DELIMITER

DELIMITER \$\$ indica al cliente SQL que no termine la definición en cada ; interno. BEGIN/END agrupa instrucciones. NEW contiene valores entrantes y OLD los anteriores. SIGNAL SQLSTATE '45000' lanza un error definido por la regla. FOR EACH ROW aplica por fila afectada. DELIMITER ; restablece el separador normal.

trg_producto_validar_insert

Detalle: BEFORE INSERT: rechaza precio ≤ 0 y stock < 0 .

trg_producto_validar_update

Detalle: BEFORE UPDATE: aplica esas mismas validaciones a nuevos valores.

trg_producto_auditar_update

Detalle: AFTER UPDATE: inserta ID, nombre anterior, precios y stocks anterior/nuevo, accion y CURRENT_USER. También se activa por descuento de venta.

trg_producto_auditar_delete

Detalle: BEFORE DELETE: conserva datos anteriores y usuario de BD.

trg_administrador_validar_email

Detalle: BEFORE INSERT: usa LIKE para comprobación básica de forma de correo.

trg_detalle_validar_insert

Detalle: Declara stock_disponible, exige cantidad/precio positivos, consulta stock y rechaza si es insuficiente.

trg_detalle_descontar_stock

Detalle: AFTER INSERT del detalle: UPDATE productos SET stock=stock-NEW.cantidad. Este UPDATE desencadena triggers de producto.

CURRENT_USER en el contexto del trigger identifica la cuenta de ejecución de MySQL; no es la cuenta autenticada de la aplicación. Los triggers se instalan después de los datos iniciales, por lo que no validan retroactivamente esos INSERT. Editar el archivo no reemplaza los triggers ya instalados. El descuento no ocurre al leer el archivo ni al ejecutar npm start; ocurre cuando MySQL recibe un INSERT de detalle con el trigger instalado.

Referencia 20. build_integracion_tests.mjs y outputs

CAPTURA 39 · build_integracion_tests.mjs

build_integracion_tests.mjs

CÓDIGO ORIGINAL

```

32
33 const wb = Workbook.create();
34 const sh = wb.worksheets.add("Pruebas de Integración");
35 sh.showGridLines = false;
36
37 sh.getRange("A1:H1").merge();
38 sh.getRange("A1").values = [["PLAN DE PRUEBAS DE INTEGRACIÓN"]];
39 sh.getRange("A2:B4").values = [["Proyecto:", "Distribuciones-v1"], ["Integrantes:",
    "_____"], ["Alcance:", "20 pruebas - Big Bang, Incremental Ascendente,
    Incremental Descendente e Híbrido"]];
40 sh.getRange("B2:H2").merge();
41 sh.getRange("B3:H3").merge();
42 sh.getRange("B4:H4").merge();
43
44 const headers = ["ID Prueba", "Módulos a integrar", "Descripción de la prueba", "Datos de entrada",
    "Resultado esperado", "Resultado obtenido (captura de pantalla)", "Estado", "Observaciones"];
45 sh.getRange("A6:H6").values = [headers];
46 sh.getRange("A7:H26").values = tests;
47
48 // Formula-driven summary so status updates remain auditable.
49 sh.getRange("A28:B32").values = [["RESUMEN", "Cantidad"], ["Total de pruebas", null], ["Aprobadas",
    null], ["No aprobadas", null], ["Pendientes / en proceso", null]];

```

Este código construye un libro de pruebas. No ejecuta una venta: los resultados del plan deben verificarse por separado.

Este script importa `fs/promises` y `Workbook/SpreadsheetFile` de `@oai/artifact-tool`. `mkdir` crea la carpeta de salida si falta. `tests` contiene veinte casos textuales de integración con identificador, módulos, objetivo, datos, resultado esperado, evidencia, estado y notas. Los grupos son Big Bang, ascendente, descendente y sándwich.

Crea un libro y una hoja, coloca títulos y datos en rangos, combina celdas y agrega fórmulas de resumen por estado. Objetos de formato asignan colores, fuente, alineación y bordes; anchos y alturas organizan la hoja. La validación ofrece estados; los formatos condicionales colorean estados; `freezePanels` mantiene encabezados; `tables.add` añade filtros y estilo de tabla.

`inspect` consulta valores/fórmulas y busca errores de hoja; `render` obtiene PNG; `fs` escribe `preview.png`; `exportXlsx` y `save` guardan el libro. Nada de esto envía POST /ventas ni comprueba inventario en MySQL. Los resultados empiezan pendientes.

`outputs/integracion_distribuciones` contiene el Excel del plan, `preview.png` y un archivo `.inspect.ndjson` de inspección (objetos JSON separados por líneas). Son evidencias/documentos generados, no recursos cargados por la aplicación. La dependencia del generador no está declarada en `BACKEND/package.json`; el generador pertenece a las herramientas de documentación.

Referencia 21. Tres recorridos completos con los archivos conectados

Crear un producto

```
FRONTEND/app.html: product-form
-> inventory.js: submit -> productPayload
-> api.js: apiRequest('/productos', POST)
-> BACKEND/src/app.js: /api
-> routes/index.js: /productos
-> productRoutes.js: requireAuth y POST /
-> authMiddleware.js -> sessionService.find
-> productController.create -> parseProduct/validateProduct
-> productModel.create -> database pool
-> MySQL: FK y trigger de validación -> INSERT
<- insertId -> JSON 201
<- inventory.js: cierra diálogo y loadInventory
```

Ejemplo de contrato de creación (categoria_id debe existir):

```
{"nombre":"Producto de práctica","tipo_corte":"Filete","categoria_id":1,"unidad_medida":"kg","precio":38000,"stock":15,"activo":true}
```

Si el botón no abre diálogo, investigar HTML/listener. Si POST responde 400, investigar payload/controlador. Si responde 500 por una categoría inexistente, investigar FK y traducción del error. No son el mismo fallo.

Iniciar sesión y luego consultar

login.js → api.startSession → authRoutes → authController → administratorModel → administradores. Con credenciales correctas, sessionService.create guarda token en Map y api.js lo guarda en sessionStorage. La siguiente petición incluye ese token; requireAuth lo busca y permite llegar al controlador. El backend no vuelve a comparar la contraseña en cada GET.

Registrar venta de 400 g

sales.js prepara 400 g; api.js envía POST /ventas; saleRoutes verifica sesión; saleController valida estructura; saleModel reserva conexión y bloquea producto; measurementService convierte a 0,4 kg. Con precio \$38.000/kg calcula \$15.200. Inserta cabecera y detalle; triggers validan/descuentan y auditan; inserta pago y confirma. De 15 kg queda 14,6 kg. Con recibido \$20.000, cambio \$4.800. La respuesta regresa a sales.js, que limpia carrito y consulta GET /ventas/ID para el recibo.

Una operación funcional atraviesa varios archivos. Una relación también puede ser un contrato de datos: producto_id en JSON debe coincidir con lo esperado por el controlador; id HTML con selector JS; ruta con fetch; campo SQL con modelo. Esas conexiones son los puntos que debes revisar cuando el profesor cambie algo.

Referencia 22. Arquitectura objetivo: cambios concretos y graduales

Conservar tecnologías y separar responsabilidades de forma progresiva:

Distribuciones/	
FRONTEND/	interfaz actual
BACKEND/	
src/	
routes/	método/URL y protección
controllers/	contrato HTTP, validación y respuesta
services/	casos de uso y reglas coordinadas
models/	consultas SQL que acepten conexión transaccional
middleware/	sesión y errores
config/	configuración de infraestructura
app.js	
server.js	
tests/	
.env.example	
package.json	
package-lock.json	
database/	
schema.sql	tablas/relaciones para instalación
triggers.sql	reglas automáticas
migrations/	cambios incrementales
seeds/	datos mínimos y de demostración separados
reset-demo.sql	reinicio explícito de pruebas
docs/	requisitos, arquitectura, datos, manuales y pruebas
scripts/	herramientas como el generador del Excel

Primero separar documentación/scripts y actualizar referencias. Después extraer la coordinación de ventas a `saleService`, dejando al modelo consultas y usando una sola conexión transaccional. No añadir otro descuento de inventario si se conserva el trigger. Luego dividir `sales.js` y centralizar políticas de cantidades/dinero. Por último, consolidar pruebas de casos normales y errores, sesiones revocables y conservación de datos históricos.

No hace falta una carpeta vacía por cada palabra técnica. Cada separación debe hacer más fácil encontrar una responsabilidad o probarla. Para defender cualquier archivo, prepara cinco respuestas: quién lo llama, qué recibe, qué valida o transforma, qué devuelve/cambia y qué otro componente necesita para hacerlo.

Galería práctica: sigue la operación

CAPTURA 40 · Guardar producto: el evento conecta la vista con la API

FRONTEND/js/inventory.js

CÓDIGO ORIGINAL

```

122     event.preventDefault();
123     const id = $('#product-id').value;
124
125     try {
126       await apiRequest(`/productos${id ? `/${id}` : ''}`, {
127         method: id ? 'PUT' : 'POST',
128         body: productPayload()
129       });
130       $('#product-dialog').close();
131       showMessage(id ? 'Producto actualizado' : 'Producto creado');
132       await loadInventory({ refreshSummary: false });
133     } catch (error) {
134       showMessage(error.message, true);
135     }
136   });
137
138   $('#product-rows').addEventListener('click', async event => {
139     const editId = Number(event.target.dataset.edit);
140     const deleteId = Number(event.target.dataset.delete);

```

product-id decide POST o PUT. productPayload recoge los campos. apiRequest transmite el objeto al controlador por HTTP.

CAPTURA 41 · Registrar venta: JSON enviado por la vista

FRONTEND/js/sales.js

CÓDIGO ORIGINAL

```

347   $('#save-sale').addEventListener('click', async () => {
348     const paymentMethod = $('#payment-method').value;
349
350     try {
351       const sale = await apiRequest('/ventas', {
352         method: 'POST',
353         body: {
354           items: cart.map(item => ({
355             producto_id: item.id,
356             cantidad: item.cantidad,
357             unidad_venta: item.unidad_venta
358           })),
359           cliente_nombre: $('#sale-customer').value.trim() || 'Consumidor final',
360           metodo_pago: paymentMethod,
361           monto_recibido: paymentMethod === 'efectivo'
362             ? Number($('#amount-received').value)
363             : cartTotal()
364         }
365       });
366
367       cart = [];
368       $('#sale-customer').value = '';

```

El navegador envía producto, cantidad y unidad. El servidor consulta el precio real; no acepta el precio del carrito como autoridad.

CAPTURA 42 · Venta: escritura y confirmación de la transacción

BACKEND/src/models/saleModel.js

CÓDIGO ORIGINAL

```

120     const [sale] = await connection.execute(
121       `INSERT INTO ventas
122         (administrador_id, cliente_nombre, metodo_pago, monto_recibido, cambio, total, estado)
123         VALUES (?, ?, ?, ?, ?, ?, 'completada')`,
124       [administradorId, clienteNombre || 'Consumidor final', metodoPago, received, change, total]
125     );
126
127     for (const item of validatedItems) {
128       await connection.execute(
129         `INSERT INTO detalle_venta (venta_id, producto_id, cantidad, precio_unitario)
130         VALUES (?, ?, ?, ?)`,
131         [sale.insertId, item.id, item.cantidad, item.precio]
132       );
133     }
134
135     await connection.execute(
136       "INSERT INTO movimientos (tipo, monto, venta_id) VALUES ('pago', ?, ?)",
137       [total, sale.insertId]
138     );
139
140     await connection.commit();
141     return { id: sale.insertId, total, monto_recibido: received, cambio: change };
142   } catch (error) {
143     await connection.rollback();
144     throw error;
145   } finally {

```

Los INSERT de detalle activan los triggers. commit confirma; rollback revierte ante error; release libera la conexión.

CAPTURA 43 · MySQL: validar detalle y descontar inventario

BACKEND/database/schema.sql

CÓDIGO ORIGINAL

```
333 CREATE TRIGGER trg_detalle_validar_insert
334 BEFORE INSERT ON detalle_venta
335 FOR EACH ROW
336 BEGIN
337     DECLARE stock_disponible DECIMAL(10,3);
338
339     IF NEW.cantidad <= 0 OR NEW.precio_unitario <= 0 THEN
340         SIGNAL SQLSTATE '45000'
341         SET MESSAGE_TEXT = 'Cantidad y precio deben ser mayores que cero';
342     END IF;
343
344     SELECT stock INTO stock_disponible
345     FROM productos
346     WHERE id = NEW.producto_id;
347
348     IF stock_disponible < NEW.cantidad THEN
349         SIGNAL SQLSTATE '45000'
350         SET MESSAGE_TEXT = 'No existe stock suficiente';
351     END IF;
352 END$$
353
354 CREATE TRIGGER trg_detalle_descontar_stock
355 AFTER INSERT ON detalle_venta
356 FOR EACH ROW
357 BEGIN
358     UPDATE productos
359     SET stock = stock - NEW.cantidad
360     WHERE id = NEW.producto_id;
361 END$$
```

BEFORE INSERT valida; AFTER INSERT descuenta. El UPDATE de productos también genera auditoría. Este código se ejecuta dentro de MySQL.

CAPTURA 44 · Impresión: se muestran los diálogos abiertos

FRONTEND/css/styles.css

CÓDIGO ORIGINAL

```
1002 @media print {
1003   body * {
1004     visibility:hidden
1005   }
1006
1007   .receipt-dialog[open],.receipt-dialog[open] * {
1008     visibility:visible
1009   }
1010
1011   .receipt-dialog[open] {
1012     position:absolute;
1013     inset:0;
1014     width:100%;
1015     max-width:none;
1016     box-shadow:none
1017   }
1018
1019   .receipt-actions {
1020     display:none
1021   }
1022
1023   .daily-report-dialog[open],.daily-report-dialog[open] * {
1024     visibility:visible
1025   }
1026
1027   .daily-report-dialog[open] {
1028     position:absolute;
1029     inset:0;
1030     width:100%;
1031     max-width:none;
```

window.print abre la impresión del navegador. Las reglas @media print determinan qué partes del documento quedan visibles.